Certifying AI-Agent Leaderboard Records: Selective Tests, Online Guarantees and the Winner's Curse

Jeffery Opoku[1,*] David Banahene[2]

[1]School of Mathematical and Statistical Sciences, University of Texas Rio Grande Valley, Edinburg, TX 78539, USA
[2]Department of Biostatistics, Robert Stempel College of Public Health and Social Work, Florida International University, Miami, FL 33199, USA
[*]Corresponding author: jeffery.opoku01@utrgv.edu

Draft of 13 October 2026

Abstract

Leaderboards for AI agents announce a record whenever a submission scores higher than every earlier one on a fixed set of tasks. A benchmark is only a sample of tasks, so some records are luck. We certify records as they arrive. The per-step test is exact on the tasks where the new system and the old record disagree, allows for the step being tested only because it is a record, and, as we prove, stays valid although the old record was itself the best so far. Three rules for the level at each step provably keep the chance of ever certifying a false record at most 5%, whatever the number of records and the dependence between them. On the 74 records of six public agent leaderboards, 20 (27%) are clearly better than the record they replaced, 13 after a correction for many tests, and the rules certify 11; a split-half check on the real data is consistent with the test separating genuine records from lucky ones. The reason is resolution: SWE-bench Verified can reliably see gaps of about 4.3 points, its typical record wins by 1.5, one point would need about 9,300 tasks, and 14 of its 135 systems could be the true best. The top score is inflated by being picked as the highest (2.2 points with equal leaders); a bootstrap correction removes about 84% of it. Tuning on public tasks is not modelled. We release `honestboard`, a Python package that computes all of this from task-by-task results.

1 Introduction

A leaderboard takes a big table of results, one row per system and one column per task, and boils it down to one score per system and one ranking. Everyone then uses that ranking: papers claim a new state of the art, companies compare products, and users pick tools. The ranking is treated as a fact, but it is really an estimate. SWE-bench Verified [24] has 500 tasks. Two agents that differ by a point and a half on those tasks actually disagree on only a few dozen of them, and which agent wins on those few depends on which tasks happened to be in the benchmark.

We call a *record step* a submission that scores higher than every earlier submission on its leaderboard, in date order; the *record it replaced* is the previous record holder. Using the task-by-task results that several leaderboards publish, we ask four questions.

1. **Records.** When a submission sets a new record, is it really better than the record it replaced, or could the gap be luck?

2. **Resolution.** What is the smallest true gap a benchmark can reliably see, and how does that compare with the gaps records are claimed by?

3. **Ranks.** How much does the top of the ranking move if we redraw the tasks, and which systems might really be the best?

4. **Winner's curse.** The top score was picked because it was the highest of many noisy scores. How much does it overstate how good the system really is, and can we correct for that?

Findings. Across 74 record steps on 5 leaderboards, 20 (27%) are clearly better than the record they replaced, using a test that allows for the fact that we only look at records because they scored higher; 13 (18%) remain after a correction for testing many records on one board; and 10 of the 65 SWE-bench records remain when code repositories, not tasks, are treated as the unit that was sampled (Section 6). The pattern follows the size of the benchmark: SWE-bench Test, with 2,294 tasks, confirms 9 of its 13 records, while the 5 boards with at most 510 tasks confirm 11 of 61. Figure 1 shows every record on four boards and Figure 4 shows why: on these boards a typical record wins by less than the smallest gap the board can reliably see. Among 135 submissions, pure luck would produce about 4.5 records; Verified has 30, so the leaderboard does track real progress, but most single records are too small for the board to confirm. Of the 14 Verified records whose leaderboard entry links a write-up (a paper, technical report or blog post, as declared by the submitter), only 1 can be told apart from the record it replaced (Table 12). Rankings wobble in the same way: two systems tie for first on Verified, the one that ranks first most often does so in only 0.35 of redraws of the tasks, and 14 of the 135 systems could be the true best. Terminal-Bench 2 has a clear leader (first in 0.80 of redraws), so the tools do not always say "noise". They work the same way on a leaderboard of plain language models: on LiveBench (318 questions) 4 of 4 records pass the basic test and 1 passes the correction for many tests, too few records to compare with the agent boards (Section 6). The winner's reported score is inflated, and a bootstrap correction removes most of the inflation when we check on tasks held back at random, with the caveats in Section 5. None of this means the records are fake or that two systems are equal: a record that we cannot confirm may still be better; the data just cannot show it.

Relation to earlier work. Careful statistics for AI evaluation is not new: error bars for language-model scores [5, 16, 32, 35], power calculations for benchmark comparisons [6], comparisons of agents that account for cost [25], run-to-run intervals for coding agents [20], the effect of the scaffolding around a model [2, 49, 54], and tests for whether a new model version got worse [28]. Table 1 sets this paper beside the five lines of work closest to it; the level rules for records that arrive one by one are the alpha-spending and online fallback rules of the sequential-testing literature, which take valid p-values as given, and what is new here is a p-value for a record step that stays valid under the selection the chain itself performs, which is what those rules need as input. Rank intervals for leaderboards are developed by Neuhof and Benjamini [37, 38], with roots in Mogstad et al. [36] and Hung and Fithian [21]; sets of possible best models in Hansen et al. [15] and Kim and Ramdas [26]; Chatbot Arena reports bootstrap intervals and rank sets [7, 8]. On the winner's curse, Andrews et al. [1], Zrnic and Fithian [56], Petrou-Zeniou and Shaikh [40] and Hoff and Tokdar [17] give corrections for a winner that has already been picked, and Bakshi et al. [3] one that adds randomness; we use the conditional method of Fithian et al. [13], Lee et al. [29] and an iterated bootstrap [10, 11], and run the hybrid of Andrews et al. [1] and the zoom interval of Zrnic and Fithian [56] as baselines. Inflation from teams privately trying many versions before submitting [44, 52] and from overfitting to a benchmark [42] are the kinds of error this paper does

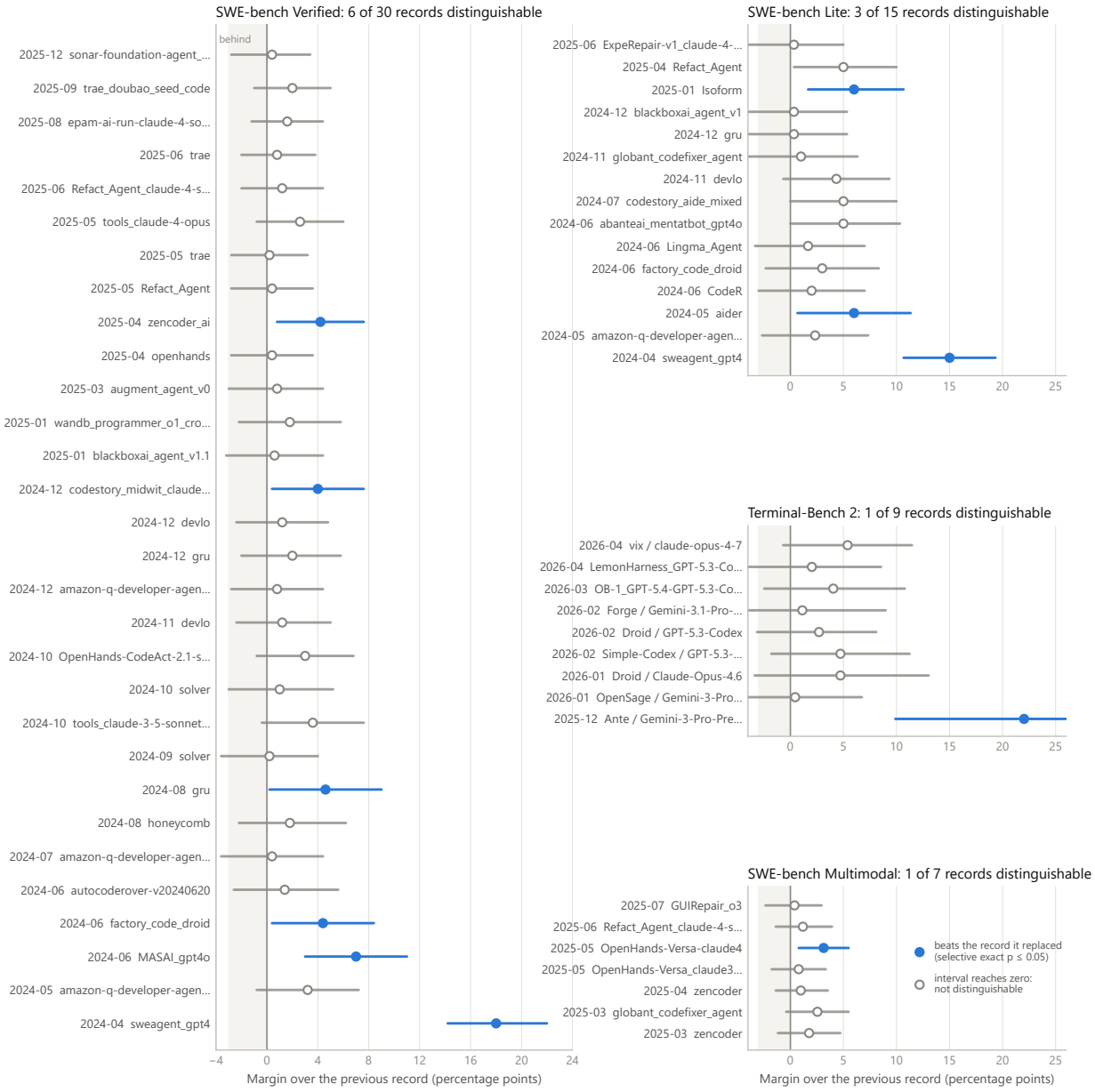


Figure 1: Every record on four public leaderboards. Each row is a submission that set a new record; the dot is how far it beat the old record and the bar is the range the gap could plausibly take (95% interval, from redrawing the tasks). Blue: the record is clearly better than the one before it (our test, $p \leq 0.05$). Gray: it is not. Newest at the top. SWE-bench Test (9 of 13 records clearly better) and Multilingual (no records under our ordering) are not shown.

Work	Setting	What it gives	What it leaves open
Liu et al. [31]	neighbouring pairs in the rankings of four SWE-bench boards	exact McNemar tests; the number of tasks a separation would need	records in time order; the selection that makes a step a record; chains of records; the winner's curse
Kotawala [27]	paired evaluations of language models	required sample size and a resolution ratio per pair, with sequential tests as a robustness check	agent boards; records
Jain [22]	simulated language-model leaderboards	sets of plausible winners from a paired bootstrap; how often the top model is truly best	models arriving over time; a test for a single record
Blum and Hardt [4], the Ladder	a holdout reused by adaptive submissions	a new best score is released only when it beats the previous best by more than a set step; a bound on the error of the released scores	per-task tests; a p-value for a record step
Foster and Stine [14]; Ramdas et al. [41]; Tian and Ramdas [48]	a stream of p-values	online error-rate rules: alpha-investing, LORD, and online fallback with family-wise error control under arbitrary dependence	they take each null p-value as valid given the past; a valid p-value for a record step, whose holder is itself a selected maximum, is what Proposition 1 supplies
This paper	every record on six public agent boards	a selective record p-value, valid for a running-maximum or a certified holder (proved); three level rules whose whole-chain guarantee then follows (proved); resolution, a set of possible best systems and a winner correction; a tool	tuning on public tasks; run noise on single-run boards

Table 1: The lines of work closest to this paper, and what each covers.

not model; an informal bootstrap analysis of Terminal-Bench 2 that grouped tasks reached a similar conclusion about its top two entries [30].

Contributions. This paper has one piece of new statistics, an audit built on it, and a tool. (i) The new statistics: a way to certify records as they arrive. We give a p-value for a record step that stays valid even though we only test a step because it is a record and even though the old record was itself the best of everything before it (Proposition 1); two rules for setting the level at each step, alpha-spending and the online fallback rule of Tian and Ramdas [48], that keep the chance of ever certifying a false record at most 5% no matter how records depend on each other (Corollaries 1 and 2; the rules and their guarantees are standard, and what is proved here is that the record p-value meets the condition they need); the same guarantee for the version in which the leader only changes when a record is certified (Corollary 3); and an online false-discovery-rate variant checked by simulation (Section 3). As far as we know this is the first certification rule for leaderboard records with a guarantee of this kind, and Table 12 shows it running over the full history of SWE-bench Verified. (ii) The audit: every record on six public agent leaderboards,

checked with that test and with a repository-level test, compared against what each benchmark can resolve; a split-half check across tasks and a run-split check across repeated runs on the real data; a language-model leaderboard run through the same pipeline; and a bound on how much reporting the best of several runs would inflate a score, from the one board that has repeated runs. (iii) Rank intervals and a set of possible best systems, with a check of how often they contain the truth, and a corrected score for the published winner, checked by simulation and on held-back tasks with error bars for the simulation itself, against the hybrid of Andrews et al. [1] and the zoom interval of Zrnic and Fithian [56]. What is proved and what is only simulated: the per-step test holds its level under its stated null (exactly, at the attainable levels, for a fixed holder), and the error over a whole chain is controlled for three rules; everything else, including LORD++, the winner correction and the set of possible best systems, rests on simulation and held-back data. Table 3 in Section 3 says which is which. (iv) `honestboard`, a small Python package with a command line and a check for new submissions,[1] and a short list of things leaderboards should report.

2 Setting

A benchmark has T tasks and K systems. System k gets a result $y_{kt} \in [0, 1]$ on task t: a pass (1) or fail (0) when each task is run once, or the share of repeated runs that passed. The reported score is the average, $\widehat{p}_k = T^{-1} \sum_t y_{kt}$. What we actually want to know is $\theta_k = \mathbb{E}[y_{kt}]$: the score system k, exactly as submitted, would get on average over all tasks of the kind the benchmark samples from. In other words, we treat the tasks as the thing that was sampled. Every system is scored on the same tasks, so the scores move together (a hard task is hard for everyone), and every comparison between two systems is a paired comparison, task by task. We estimate how the scores co-vary by $\widehat{\Sigma}_{ij} = \{T(T-1)\}^{-1} \sum_t (y_{it} - \bar{y}_i)(y_{jt} - \bar{y}_j)$. Where tasks come in groups (the code repository they belong to) we also use a version of this that sums within groups, and a bootstrap that resamples groups.

What the numbers are about. Everything below is about θ_k in that sense: the score a system would get on more tasks of the same kind as the benchmark's (the same repositories, the same kinds of problems, the same scoring). It is not about repositories the benchmark does not contain, about real-world use, or about a system after it has been tuned further. A reported score has three parts: the true score θ_k, luck in which tasks were drawn, and inflation from development, because teams tune their systems, and sometimes train them, with the public tasks in view [44, 52]. With one run per system we can only measure the second part. Table 2 says which part each of our procedures handles. Inflation from development and the randomness of an agent from one run to the next are not modelled; both push reported scores up, so a winner may be more inflated than our correction removes. Section 6 puts a number on one form of development inflation, reporting the best of several runs, using the Terminal-Bench submissions that ran the whole benchmark more than once.

3 Methods

Record test. Let a be the record holder and b a new submission with a higher score, $\widehat{p}_b > \widehat{p}_a$. Tasks that both solved or both failed tell us nothing about which is better, so we look only at the tasks where they disagree: n_b tasks solved only by b and n_a solved only by a, with $n = n_a + n_b$ in

[1] Version 0.1.0, MIT licence: `https://github.com/jopoku16/honestboard`, with the built boards, every result file, the scripts behind each table and figure, and the manuscript source.

Where a reported score can be wrong	What handles it here	Status
Luck in which tasks were drawn	paired tests, rank intervals, set of possible best systems, resolution, winner correction	modelled
The agent giving different answers from one run to the next	nothing (single runs); measured on Terminal-Bench from its five runs per task	not modelled
Development: versions tried, tuning and training on the public tasks	nothing; the effect of reporting the best of several runs is bounded from Terminal-Bench's repeated runs	not modelled

Table 2: What the procedures in this paper handle, and what they do not.

total. If the two systems were equally good, each of these n tasks would be a coin flip, so the natural test asks whether n_b wins out of n is more than a fair coin would give. This is the binomial (sign) test, the exact form of McNemar's test [33]. There is a catch: we only look at this step *because* b won more of the disputed tasks ($n_b > n_a$), and that alone makes the count look impressive. So we run the test given that fact: with $X \sim \mathrm{Bin}(n, \frac{1}{2})$ the number of heads in n fair coin flips, the *selective* p-value is

$$p_{\mathrm{sel}} = P(X \geq n_b \mid X > n/2) = \frac{P(X \geq n_b)}{P(X > n/2)}.$$

In words: among all the ways a fair coin could have given b the majority, how often would it have given b at least this many wins? A p-value is the chance of seeing a result at least this extreme if the two systems were really equal; a small one (we use 0.05, one in twenty) means luck is an unlikely explanation. If the holder were fixed in advance and the null held (on each disputed task b is no likelier than a to be the one that passes), p_{sel} would be a valid p-value exactly; Proposition 1 below shows it stays valid when the holder is itself the best of everything before it. For odd n it equals the usual two-sided exact p-value. For even n it is larger, because $P(X > n/2) = \{1 - P(X = n/2)\}/2 < \frac{1}{2}$: the two-sided test then rejects too often among record steps (with $n = 44$ its conditional size is 5.5%), and a one-sided test at level α rejects about 2α of them. We call a step significant when $p_{\mathrm{sel}} \leq 0.05$; the two-sided and one-sided tests at 5% are reported as sensitivity checks. Section 5 checks by simulation that the test rejects no more than 5% of true-null steps, under two nulls and by the step's position in the chain, and repeats the test on split halves of the real boards. When a board has many record steps, some will pass at 5% by chance; Holm's correction [18] raises the bar so that the chance of *any* false pass on one board stays at 5%. For Terminal-Bench, where each y_{kt} is a rate rather than a pass or fail, we use the sign-flip version: randomly flip the sign of each task's difference many times (2×10^5 sign patterns, the same patterns in the audit and in the certification, so that a p-value near 0.001 has a Monte Carlo standard error of about 10^{-4}) and compute $P(S \geq s_{\mathrm{obs}} \mid S > 0)$ for the sum S; this one is approximate. Tasks from the same code repository are not independent (an agent that handles one repository well tends to handle its other tasks well too), so the task-level test is the optimistic one and the repository-level test below the cautious one; Section 5 measures the difference.

Submissions sharing a date. Under the best-first ordering (described below) the challenger at a record step is the best of all submissions that share its date. That is a second kind of selection the record event does not cover: in a simulation where no system is better than any other and 20 submissions share a date, the per-step test rejects about 10% of record steps instead of 5%. We therefore multiply the p-value of a record step by g, the number of submissions on the

challenger's date, the standard Bonferroni fix for picking the best of g (the package's tests include that simulation); a submission with a date of its own has $g = 1$. On the real boards 19 of the 74 record steps have g between 2 and 4; without the correction 20 steps are significant, with it 20.

Repository-level record test. For boards that label each task with its code repository we also run a repository-level test: add up the per-task differences within each repository, then randomly flip the sign of each repository total (all 2^G patterns when there are $G \leq 14$ repositories). Its selective form conditions on a positive sum in the same way. It is valid as long as the repository totals are equally likely to be positive or negative under the null; with only a dozen repositories it has little power, which is why the task-level count stays the headline and the repository-level count sits beside it as the cautious one.

Ordering submissions that share a date. Which steps count as records depends on how we order submissions with the same date. The main analysis orders them best first: this gives the fewest and biggest steps, never invents a record that was not on the leaderboard, and is the most generous reading for the board. We also report worst first, which gives the most steps but can create records that never actually appeared, so we treat it only as a bound. The best-first challenger is the best of its date, which is the selection the per-date factor above corrects for.

Certifying records as they arrive. A leaderboard has to decide about each record when it comes in, without knowing how many records will follow, so Holm over the finished chain is only something you can do looking back. We give a procedure for the live setting. For the i-th record step, with holder a and challenger b, let $p_i = p_{\text{sel}}$ be the selective p-value of the record test. With a fixed holder it is a valid p-value given that the step is a record, because the record event is exactly $\{X > n/2\}$. A holder that is the best so far got there partly by luck, so its results are tilted upward; this works in our favour, making p_i if anything too cautious, and it is a theorem, not only a simulation.

> **Proposition 1.** Let the outcome vectors of the K systems be independent across tasks, and let the *conditional task-wise dominance null* hold: for each task, given the task and the outcomes of every system other than a and b (earlier and later), the probability that b passes and a fails is at most the probability that a passes and b fails. (Equality is the null of the exact test, no task-level advantage for b; the null implies $\theta_b \leq \theta_a$ but is not implied by it.) Let A be any event that does not involve the outcomes of b and that, given the outcomes of the other systems, is increasing in the outcomes of a task by task; the event that a holds the record among the earlier submissions, which depends on a only through its total, is one. Then, conditional on A, on the record event $\{n_b > n_a\}$, on the discordant set and on the outcomes outside it, $P(p_i \leq u) \leq u$ for every $u \in [0, 1]$.

In plain terms: under the stated null and with tasks independent, knowing that the holder is "the best so far" (or anything else that only gets more likely as the holder does better) can only make the p-value more cautious, never less. The proof has three steps: under the null the challenger's win count on the disputed tasks behaves like at most a fair coin; knowing the holder did well pushes that count further down; and a count pushed down gives a larger p-value.

Proof. Fix the discordant set D with $|D| = n$, the outcomes of a and b outside D and the outcomes of every other system. By independence across tasks, given these the n tasks of D are independently b-only with probabilities at most one half, so the law of n_b is dominated in the likelihood-ratio order by $\text{Bin}(n, \frac{1}{2})$: Bernoulli laws are log-concave, and the likelihood-ratio order is

preserved under convolution of log-concave laws [43, Section 1.C]. Within D, a passes exactly the tasks that are not b-only, so A is a decreasing event in the vector of b-only indicators, and by Efron's monotonicity theorem for independent variables with log-concave laws [9], $P(A \mid n_b = k, \dots) = g(k)$ is a decreasing function of k (for the event that a holds the record, an indicator of $\{k \le \text{constant}\}$). Conditioning on A multiplies the law of n_b by g, which moves it downward in the likelihood-ratio order; conditioning further on $\{n_b > n/2\}$ restricts both laws to the same set and preserves the order. Hence $n_b \mid A, \{n_b > n/2\}$ is stochastically at most $X \mid \{X > n/2\}$ with $X \sim \mathrm{Bin}(n, \frac{1}{2})$; p_i is a decreasing function of n_b, so $P(p_i \le u) \le P\{p(X) \le u \mid X > n/2\} \le u$. □

The bound is exact at the levels the discrete p-value can actually take when the null holds with equality, the holder is fixed and $g = 1$, and cautious otherwise. We also check the proposition by brute force in the package's tests: on small boards we list every possible outcome, separately for each number of disputed tasks, for holder events that do and do not depend only on the total, including "the holder was certified and a later challenger arrived", and we include events that go the wrong way as negative controls, which must break the bound and do. The proposition also covers the certified baseline, through a simple monotonicity property of the selective p-value.

> **Lemma 1.** p_{sel} decreases when a task won by the challenger is added to the discordant set and increases when a task won by the holder is added.

In words: more wins for the challenger make the step look stronger, more wins for the holder make it look weaker, which is what one would hope. The proof is a short Pascal-rule computation (Appendix C). Whether a was certified against its own reference, and whether no later submission has been certified against a, both depend on a's outcomes only through counts of this kind, so by Lemma 1 the event "a is the certified holder when b arrives" is increasing in a's outcomes given the other systems, and Proposition 1 applies to it.

So the proposition covers both baselines: the leaderboard holder, which is simply the best so far, and the certified holder, which earned its place by passing an earlier test. Its null is the conditional task-wise dominance null, which asks more than $\theta_b \le \theta_a$: when b is a tweaked copy of an earlier system c, b is likelier than a to pass exactly where c passed, the per-task probabilities sit on both sides of one half, and the proof says nothing, although every record may still be luck. A rejection is therefore evidence against a dominating b task by task, the same reading as for McNemar's test; validity under the weaker null $\theta_b \le \theta_a$ alone is not proved, and Section 5 simulates the case where the two differ. How strict the test is at the i-th record (its level α_i) is set by one of three rules. *Alpha-spending:* $\alpha_i = 6\alpha/(\pi^2 i^2)$, so that $\sum_i \alpha_i = \alpha$. Think of α as a budget of allowed error, 5% in all, handed out in shrinking pieces to the records as they arrive: the first record gets about 61% of it, the second about 15%, and so on, so the pieces never add up to more than the budget. Its guarantee follows from Proposition 1 and the union bound (the chance that at least one of several things happens is at most the sum of their chances), with no independence assumed.

> **Corollary 1.** Let the conditions of Proposition 1 hold for every pair that forms a record step on the leaderboard baseline, and let the i-th record step be tested at level α_i with $\sum_i \alpha_i \le \alpha$. Then, for any submission calendar fixed in advance (same-date submissions handled by the per-date factor), any number of submissions and any dependence between steps, the probability that some record step at which the conditional task-wise dominance null holds is certified is at most α.

In words: hand out the error budget in pieces that add up to α, and the chance that any false record is ever certified is at most α, however long the chain gets and however the steps depend on one another. *Proof.* Take first a submission order that does not depend on outcomes (distinct

dates). Let $E_i(a,b)$ be the event that the i-th record step has holder a and challenger b. It is the intersection of the record event $\{n_b > n_a\}$ with an event that does not involve b, depends on the outcomes of a only through its total score and is increasing in it: a scored higher than every submission before it, no submission between a and b scored higher than a, and the number of record steps before a, which is determined by the submissions before a, is $i-1$. Proposition 1 applies with A that event, so $P(p_i \le \alpha_i \mid E_i(a,b), \ldots) \le \alpha_i$ whenever the null holds for (a,b); summing over the disjoint events $E_i(a,b)$ gives $P(\text{a null pair is certified at step } i) \le \alpha_i$, and the union bound over i gives the result. Under the best-first ordering the challenger is the best of the g submissions on its date δ, a selection the record event does not cover, and the set of submissions between a and b depends on b's score; the per-date factor handles both. Fix a holder a, a date δ with g members and an index i, and let $A_i(a,\delta)$ be the event that a holds the record among the submissions of dates before δ, with $i-1$ record steps before it; it involves only earlier dates, so it is free of every member's outcomes, and it is increasing in a's total. A member d can be certified at step i only if d beats a and $p_d \le \alpha_i/g$, so Proposition 1, applied to the pair (a,d) given $A_i(a,\delta)$ and $\{d \text{ beats } a\}$, gives $P(d \text{ is null and certified at step } i) \le (\alpha_i/g)\, P\{A_i(a,\delta) \cap \{d \text{ beats } a\}\} \le (\alpha_i/g)\, P\{E_i(a,\delta)\}$, where $E_i(a,\delta) = A_i(a,\delta) \cap \{\text{the best of } \delta \text{ beats } a\}$ is the event that step i falls on date δ with holder a. Summing over the g members gives $\alpha_i P\{E_i(a,\delta)\}$; the events $E_i(a,\delta)$ are disjoint over (a,δ), so $P(\text{a null pair is certified at step } i) \le \alpha_i$, and the union bound over i completes the proof. □

Corollary 1 needs two things to be settled before the challenger's results are looked at: the level used at step i and the event that put the holder in place. Both hold for the leaderboard baseline, because how many records came before a is decided by the submissions before a. For the certified baseline, how many tests came before a challenger depends on how many submissions beat the certified holder, which depends on the holder's own results, so levels that count tests are not settled in this sense; levels that count calendar dates since the holder arrived are.

> **Corollary 3.** Under the conditions of Proposition 1 for every tested pair, with the best-first ordering and the per-date factor, the certified-baseline procedure with the tenure-indexed levels $\alpha_{j,m} = \alpha\gamma_j\gamma_m$, where j is one plus the number of certifications so far and m the number of distinct submission dates since the certified holder's date, has a super-uniform p-value at every test and certifies some null pair with probability at most α, for any dependence.

In words: on the certified baseline, set the level from two counters that the challenger's results cannot touch, how many certifications have happened and how many dates have passed since the certified holder arrived; the whole-chain guarantee then holds. *Proof.* With these levels the event that a is the certified holder when b arrives, that a was certified against its own reference and that no later submission has been certified against a, involves a only through counts of the kind in Lemma 1 at levels fixed by the calendar, so it is increasing in a's outcomes and does not involve b; Proposition 1 gives per-step validity. Given the conditioning, j is determined by the submissions before the holder together with the holder event, and m by the calendar, so the level is fixed; each pair (j,m) is used by at most one date, the per-date correction handles the members of that date as in Corollary 1, and the levels used over any realised chain sum to at most $\alpha \sum_j \gamma_j \sum_m \gamma_m \le \alpha$. □

The tenure rule spends a little budget on every date since the holder arrived, whether or not anything was tested that day, which costs power on long chains. The spending and fallback rules that count tests instead have no such proof on the certified baseline, per step or over the chain, because the number of tests carries information about the holder's score; they are checked by simulation only.

Online fallback: $\alpha_i = 6\alpha/(\pi^2 i^2) + \alpha_{i-1}\,\mathbf{1}\{\text{record } i-1 \text{ was certified}\}$. In words: a record that passes passes its level on to the next record, so a run of genuine records builds up budget. The rule is the fixed-sequence fallback procedure [51] in the online form of Tian and Ramdas [48], who prove its family-wise error control for arbitrarily dependent p-values; it is never stricter than alpha-spending at any step, and Corollary 2 records that their guarantee carries over here once Proposition 1 supplies the per-step p-value and the per-date factor handles the ties.

> **Corollary 2.** Under the conditions of Corollary 1, the online fallback rule certifies some record step at which the conditional task-wise dominance null holds with probability at most α.

In words: the first false record to be certified can only have inherited budget from genuine records just before it, and each genuine record's budget can be inherited only once, so the budget available to a first false certification adds up to at most α. *Proof.* Let step i be the first null step certified. Every step in the run of consecutive certified steps ending at $i-1$ is then non-null, so the level used at i is at most $L_i = \alpha \sum_{k \in R_i} \gamma_k$, with $\gamma_k = 6/(\pi^2 k^2)$ and R_i the maximal run of non-null steps immediately before i together with i itself. R_i is determined by the identities of the earlier pairs, which are fixed by the conditioning of Proposition 1 on $E_i(a, b)$ and the earlier submissions, so Proposition 1 gives $P(\text{step } i \text{ is the first null step certified}) \le \mathbb{E}[\mathbf{1}\{\text{step } i \text{ is null}\}\, L_i]$; with the per-date factor each member of the date is tested at L_i/g, and the per-date argument of Corollary 1 applies with L_i in place of α_i. The runs R_i of distinct null steps are disjoint, so $\sum_{i \text{ null}} L_i \le \alpha \sum_k \gamma_k = \alpha$ on every realisation of the chain, and summing over i gives the result. □ *LORD++* [41]: an online false-discovery-rate rule, which controls the expected share of certified records that are false rather than the chance of any false one, with level $\alpha_i = \gamma_i W_0 + (\alpha - W_0)\gamma_{i-\tau_1} + \alpha \sum_{j \ge 2} \gamma_{i-\tau_j}$, where τ_j is the index of the j-th certified record, $W_0 = \alpha/2$, and γ_j is any sequence with $\sum_j \gamma_j \le 1$; we use $\gamma_j = 6/(\pi^2 j^2)$, the same sequence as the spending rule, because the sequence published with LORD++ is built for streams of thousands of tests and leaves almost no budget for a chain of thirty records. Each certified record earns budget back, so late records stay testable; LORD++ starts with half the spending rule's budget and overtakes it once records are certified. Its guarantee is proved for independent p-values, and ours share tasks: alpha-spending and online fallback are the rules with the guarantee, and LORD++ is an alternative checked by simulation; Javanmard and Montanari [23] give a more cautious variant for dependent p-values, which we have not implemented. There are two choices for what to test against: each leaderboard record against the record it replaced, or every submission that beats the last *certified* record against that record, so that the certified leader only changes when a record is certified, as in the Ladder [4]; Corollary 3 covers the second. A normal-theory alternative that accounts for both selections at once (the polyhedral lemma applied to the event that b is a record and a was the holder) is described in Appendix D; it needs a continuity correction on discrete scores and has no finite-sample guarantee, so it is reported only as a comparison. Table 3 lists, for every procedure in the paper, what is proved and what rests on simulation.

Lag. For each record we ask how far back in the chain of records one must go before the new record is clearly better than that older record (same test, no correction). A lag of one means the record is clearly better than the one it replaced; a lag of three means it is clearly better than the record three steps back but not the two in between.

Resolution. Resolution means the smallest true gap a benchmark can reliably see. Let s be the typical (median) standard deviation of the per-task differences between systems next to each

Procedure	Baseline	Guarantee	Where
Selective record test (with the per-date factor)	either	holds its level given the record event under the conditional task-wise dominance null, tasks independent (exact at attainable levels for a fixed holder with $g = 1$); conservative for a running-maximum or certified holder and for $g > 1$	Prop. 1, Lemma 1
Alpha-spending	leaderboard	family-wise error $\leq \alpha$ over the chain, any dependence	Cor. 1
Online fallback	leaderboard	the same, with a level never below spending's	Cor. 2
Tenure-indexed spending	certified	family-wise error $\leq \alpha$ over the chain, any dependence	Cor. 3
Spending, fallback	certified	none (the test count depends on the holder's outcomes); by simulation	Sec. 5
LORD++	either	false-discovery rate proved for independent p-values; dependent here, by simulation	Sec. 5
Holm over the realised chain	leaderboard	retrospective, adaptive family; by simulation	Sec. 5
Polyhedral p-value	leaderboard	none; continuity-corrected, by simulation	App. D
Hybrid and zoom intervals	–	coverage under normality with known covariance	Sec. 3
Iterated bootstrap	–	none; by simulation and held-out tasks	Sec. 5
Confidence set of the best	–	retains a truly best system at about the nominal rate; by simulation	Sec. 5

Table 3: What is proved and what is simulated, for every procedure in the paper.

other in the ranking, among the ten highest-scoring systems. With $z = \Phi^{-1}(0.975) + \Phi^{-1}(0.80)$, the paired test at level 0.05 (two-sided; the selective record test has the same threshold up to the even-n correction) detects a true gap of MDD $= z\, s/\sqrt{T}$ (the minimum detectable difference) with 80% power, meaning it will catch a gap that size four times out of five, and catching a gap Δ with that power takes $(zs/\Delta)^2$ tasks. Redrawing the tasks with a bootstrap gives an interval for both numbers. The smallest margin the test will call significant for a typical pair is about $1.96\, s/\sqrt{T}$, roughly 0.7 of the MDD. This is a normal approximation [6, 27]; Section 5 checks it by simulation.

Rank intervals and top-rank frequency. We redraw the tasks with replacement many times, recompute every score and rank each time (tied systems share the better rank), and report the range that holds the middle 95% of each system's ranks. The top-rank frequency of a system is the share of redraws in which it has the top score, with ties split equally. It is a frequency across redraws of the tasks, not the probability that the system is best. Section 5 shows that the intervals contain the true rank when the true gaps are as big as the observed ones, but not when the leaders are truly tied, so the set below is the right statement to make about who could be best.

Set of possible best systems (confidence set of the best). Following multiple comparisons with the best [19], as adapted to paired bootstraps by Jain [22], we keep every system that no rival beats by more than a margin set to cover all comparisons at once. Let $\widehat{s}_{ij}$ be the standard error of $\widehat{p}_j - \widehat{p}_i$ from $\widehat{\Sigma}$ and $\widehat{p}^*$ scores from one redraw of the tasks. For each system i let $M_i = \max_{j \neq i}\{(\widehat{p}^*_j - \widehat{p}^*_i) - (\widehat{p}_j - \widehat{p}_i)\}/\widehat{s}_{ij}$, the furthest any rival moves ahead of i in a redraw, in standard-

error units. System i stays in the set if $\widehat{p}_j - \widehat{p}_i \le c\,\widehat{s}_{ij}$ for all j, with c the largest over i of the $(1-\alpha)$ quantile of M_i. A system that is truly best then stays in the set about $1-\alpha$ of the time, which is all the set promises; an earlier draft used the quantile of $\max_i M_i$, an all-pairs bound that gives larger sets, and we report both. The guarantee is approximate and holds only for large samples, and in our simulations the set is on the cautious side. The set says who cannot be ruled out, not who is likely to be best.

The winner's curse. Let $w = \arg\max_k \widehat{p}_k$ be the system with the top score. Its reported score $\widehat{p}_w$ is too high on average, because w was picked for having the highest score and part of a highest score is good luck. This is the winner's curse. We consider two corrections.

Conditional (polyhedral) method. Treat the scores as normal, $\widehat{p} \sim N(\theta, \widehat{\Sigma})$, with $\widehat{\Sigma}$ known. The event that w wins is $\{\widehat{p}_j - \widehat{p}_w \le 0,\ j \ne w\}$. By the polyhedral lemma [13, 29], given this event and the part of $\widehat{p}$ unrelated to $\widehat{p}_w$, the winner's score follows a normal distribution with mean θ_w and variance Σ_{ww} cut off to an interval $[V^-, V^+]$ whose ends depend only on the other systems' scores and on $\widehat{\Sigma}$. Inverting that cut-off normal distribution gives a median-unbiased estimate and a confidence interval for θ_w that remain valid after the winner was picked. It is exact under normality, but it breaks down when the winner ties its runner-up, and nearly so when the margin is small, because the observed score then sits right at the edge of the cut-off and the interval runs off to infinity; two of our six real boards have an exact tie at the top.

Baselines. Two published corrections run alongside ours. The hybrid of Andrews et al. [1] fixes the runaway-interval problem by first drawing a wide interval that covers every system's true score at once with confidence $1-\beta$, and then applying the conditional method inside that interval; the result can never run off to infinity, and the conditional level is adjusted to $(\alpha-\beta)/(1-\beta)$ so that overall coverage is still at least $1-\alpha$. Its estimate is median-unbiased under the hybrid distribution. We use $\beta = \alpha/10$. The zoom correction of Zrnic and Fithian [56], in its simplest, locally simultaneous form [55], works in two stages: spend a share ν of the error budget on intervals for every system that all hold at once, keep the systems that could plausibly be the winner (those whose upper limit reaches the highest lower limit), then spend the remaining $\alpha - \nu$ on a joint interval over that smaller set alone; it corrects the interval, not the point estimate, and we use $\nu = \alpha/10$. Both are exact when the scores are normal with known covariance; the joint quantiles are computed by Monte Carlo from $\widehat{\Sigma}$. Hoff and Tokdar [17] is not implemented.

Iterated bootstrap. The bootstrap builds a stand-in world in which the observed scores are the truth, by redrawing the tasks with replacement. In that bootstrap world we know every system's true score, so we can measure directly how much a winner's score is inflated by being the winner. Draw a task resample, let w^* be its winner, and set $\Delta^* = \widehat{p}^*_{w^*} - \widehat{p}_{w^*}$, how far the bootstrap winner beat its own bootstrap truth. The plug-in correction is $\widehat{p}_w - \widehat{b}_1$ with $\widehat{b}_1 = \mathbb{E}^*\Delta^*$, the average of that gap [11, 12]. It does not remove all the bias, because the observed scores of the leaders are more spread out than their true scores, so being the winner is a smaller achievement in the bootstrap world than in reality. A second round of bootstrap, run inside each resample, estimates the bias that remains: $\widehat{\theta}_w = \widehat{p}_w - 2\widehat{b}_1 + \mathbb{E}^*\widehat{b}^*_1$, where $\widehat{b}^*_1$ is the plug-in bias computed from each outer resample. The interval is $\widehat{\theta}_w$ minus the quantiles of the centred Δ^*. It works for ties and when repositories rather than tasks are resampled. Neither correction comes with a general theory here: the conditional method is exact only when the scores are normal with known covariance, and the iterated bootstrap is justified by how well it does in Section 5, not by a theorem; taking a maximum is not a smooth operation, so the bootstrap is not guaranteed to work near ties.

4 Data

We use the task-by-task results that the leaderboards themselves publish. For SWE-bench [24, 39, 45, 53] we use the submission records in the public `experiments` repository [46]: for each submission, the list of tasks it solved, matched against the official task list so that tasks nobody solved are counted too. Verified has 135 usable submissions out of 182 on 500 tasks; Lite has 84 on 300, Test 24 on 2,294, Multimodal 12 on 510, and Multilingual 14 on 300. Each submission is one run, dated by the date in its folder name; 11 of the 14 Multilingual submissions share one date, so which steps count as records there depends on the ordering rule, and under our main rule it has none. Two Verified submissions tie exactly for first (396 of 500 tasks, disagreeing on 36 tasks, a tie that Liu et al. [31] also report), and two Multimodal submissions tie for first; we name both and call the one that ranks first more often across redraws the leader.

The 47 Verified submissions with no task-by-task file are dated 2025-07 to 2026-02 and report only an overall score. Their median score is 61.0% against 53.2% for the submissions we could use, so they are not a random sample of the board. Their highest score is 76.8%, below the record of 79.2%, and by overall score none of them would ever have held the record: the chain built from all 182 submissions has the same 30 steps. So none of them could have been the record that one of our tested records replaced, but they are missing from the ranks, from the set of possible best systems and from the pool the winner was picked from, and we do not know which tasks they solved.

For Terminal-Bench 2 [34, 47] we use the result file of each job in the leaderboard dataset. Every run (trial) named in a file is counted once: runs listed with reward 1 pass, runs listed with reward 0 fail, and runs that appear only in the error lists (agent errors, timeouts) count as failures. The per-task score is the share of that task's runs that passed. Of 75 submissions, 2 whose job files hold no per-task results, 1 with a job labelled as benchmark version 2.1 and 5 with missing tasks or dates are dropped, leaving 67 on 89 tasks. The leaderboard is meant to run five trials per task, but 26 submissions pool several jobs, and across the kept submissions the number of runs on a task ranges from 1 to 10; 60 submissions have exactly five everywhere. Most job files do not say which benchmark version they ran; we trust the dataset's folder (the 2.0 leaderboard).

For the comparison with a leaderboard of plain language models rather than agents we use LiveBench [50], which scores each question automatically, from the per-question file the leaderboard is computed from (the `livebench/model_judgment` dataset, April 2025 version: 195 models and 494 questions in three categories, coding, instruction following and language). Its questions change over time, so different models were judged on different sets; we keep the 318 questions that at least half of the models answered and the 94 models that answered every one of them (101 dropped, most of them partial re-judgings), average the 571 cases where one model was judged twice on one question, and date a model by its earliest judgment in the file, from 2024-06 to 2025-03; 8 models share the first date. Scores are credit between 0 and 1, 77% of them exactly 0 or 1, and for the grouped test the groups are its 6 task types. The data and code to rebuild every board are in the project repository.

5 Validation

5.1 Does the record test reject as often as it claims?

A test at the 5% level should wrongly call a lucky record "clearly better" no more than 5% of the time. We check this on simulated boards built to look like SWE-bench Verified: each task gets a difficulty from the real pass rates of the 30 strongest systems (standard deviation 2.40 on the logit scale), system k passes task t with probability $\text{sigmoid}(a_k + L_t)$, and the outcomes of different

systems on the same task are linked through a Gaussian copula with correlation $\rho = 0.29$, tuned so that the task-by-task spread between two equally able systems (0.34) matches the real board (0.34); real systems share base models and scaffolding, so they agree on far more tasks than shared difficulty alone would give. Two null worlds, in which no system is really better than another, are simulated (Table 7). In the *global null*, 40 equally able systems arrive one after another, so every record is luck and the holder is always the running maximum (600 boards per size). In the *selected-step null*, holder and challenger are equally able and the holder is not inflated; we keep the pairs in which the challenger scored higher (20,000 pairs per size), which is the record condition. At 500 tasks the one-sided test at 5% wrongly rejects 8.0% of selected steps, close to the 2α predicted above, the two-sided test 3.9% and the selective test 3.7%. Under the global null the selective test's rate is 0.5% averaged over the chain, but the average hides where in the chain the step is: the first step, where the holder is simply the first submission and so only mildly lucky, rejects 4.0% one-sided and 1.0% selective, and later steps, against a holder that is the best of many, almost never (Supplement, Table S1). The selective test is at or below 5% given the record event by construction, and the simulation agrees in every case, also with the model fitted to Lite, Terminal-Bench and LiveBench (Table 7: with the model fitted to Lite the selective test wrongly rejects 4.1% of selected null steps at 500 tasks, fitted to Terminal-Bench 3.3% and fitted to LiveBench 4.0%); the two-sided test differs from it only through the even-n effect, and the one-sided test rejects too often. The chance that Holm wrongly passes at least one step on a global-null board is 0.7%. A second simulation checks the resolution formula: at the formula's minimum detectable difference, the selective test catches the gap 0.78 of the time at 500 tasks (Supplement, Table S2), against a target of 0.80.

A split-half check on the real boards asks whether the test actually separates genuine records from lucky ones (Table 4). For every record step we split the tasks into two random halves 200 times, run the test on one half, and record whether the challenger still beats the holder on the other half. Steps that pass on the first half (19% of splits, against 3% under the pure-luck reference below) beat the holder on the second half 0.92 of the time and pass the test there 0.67 of the time, against 0.79 and 0.08 for steps that did not pass on the first half. Those raw figures need a reference point: every step is a record on the full data, so a challenger that did well on one half must have done less well on the other, and even a pure-luck record beats the holder on the second half more often than not. We therefore simulate that pure-luck reference for each step with its own number of disputed tasks (fair-coin signs, conditioned on the record event on the full data, then the same split): it gives 0.47 for first halves that pass and 0.74 for those that do not. Steps that pass repeat far above that reference, and steps that do not pass repeat at about it, on every board, which is what a test that separates genuine records from lucky ones should produce (on SWE-bench Test even the steps that do not pass sit above the reference, Table 4, the same message as the run split below: small real gaps the board cannot confirm); the pooled figures are dominated by SWE-bench Test, and Multimodal and Terminal-Bench contribute few passing splits.

Terminal-Bench allows a second kind of replication, across runs rather than across tasks: most submissions ran every task five times, so for each of its 9 record steps we split each task's runs at random into two parts, half the runs rounded down against the rest (two of five for most tasks), 200 times, run the test on the first part and check the second; a pure-run-noise reference redistributes each task's runs between the two systems at random, keeps the draws that form a record on the full data, and applies the same split. Record steps that pass on the first part beat the old record on the second 1.00 of the time (reference 0.58), and the steps that do not pass 0.91 of the time (reference 0.78). The second number is the informative one, with a caveat: it pools the 8 records that do not pass on the full data, and for 6 of them the replication rate sits clearly above the reference, while the two smallest margins sit at or below it. For those records what hides them is not the agent's

Board	Steps	Half 1 significant			Half 1 not significant		
		share of splits	replicates	null reference	share of splits	replicates	null reference
SWE-bench Verified	30	10%	0.85	0.48	90%	0.76	0.74
SWE-bench Lite	15	13%	0.87	0.45	87%	0.78	0.73
SWE-bench Test	13	63%	0.99	0.49	37%	0.88	0.75
SWE-bench Multimodal	7	5%	0.71	0.47	95%	0.80	0.73
Terminal-Bench 2	9	10%	0.78	0.43	90%	0.80	0.73
All boards	74	19%	0.92	0.47	81%	0.79	0.74

Table 4: Split-half check of the record test on the real boards, 200 random halves per record step. Share of splits: how often the test on half 1 passes. Replicates: share of those splits in which the challenger still beats the holder on half 2. Null reference: the same quantity simulated for each step with its own number of disputed tasks under fair-coin signs, conditioned on the record event on the full data.

run-to-run randomness but the draw of 89 tasks, which a run split cannot vary, a conclusion by elimination within the model of Section 2; the 1 record that passes on the full data replicates on every split (1.00).

Tasks from the same code repository move together, and the task-level test treats them as if they were independent. Table 6 simulates the selected-step null with a system-by-repository effect on Verified's own 12 repositories, one of which holds 231 of the 500 tasks: each system gets a random shift $u_{kg} \sim N(0, \tau^2)$ to its logit in repository g, on top of the shared difficulty and the copula, so a system can be especially good or bad at one repository. At $\tau = 0.25$ the task-level selective test wrongly rejects 9.6% of null record steps and at $\tau = 0.5$ 24.9%, while the repository-level test stays at 3.1% and 3.4%. The data do not say exactly how big τ is. A direct estimate from the per-repository residuals of the 30 strongest systems is zero, but the typical ratio of the repository-level to the task-level standard error for neighbouring top-10 pairs on the real board is 1.13, against 0.88 if tasks were independent with these repository sizes (the repository-level estimator comes out too small when one repository dominates) and 1.01 and 1.32 at $\tau = 0.25$ and 0.5, which points to a τ in that range. So the task-level counts are optimistic where repositories matter, by an amount the data leave uncertain, and the repository-level counts in Section 6 are the cautious ones.

Tuning on the public tasks is outside our model, but its simplest form can be priced (Table 5). Suppose a team tries V equally able versions of its system on the benchmark and submits only the best one. The record test allows for one challenger having scored higher, not for the best of V, so under the selected-step null it wrongly rejects 4.9% of record steps at $V = 2$, 8.2% at $V = 5$ and 12.4% at $V = 10$, and the share of submissions that set a record rises from 48% to 90%. If the team declares V, multiplying the p-value by V (the Bonferroni correction, $\min(1, Vp_{\text{sel}})$) restores the level (1.8% at $V = 5$, 1.5% at $V = 10$); an exact test for the best of V would need the task-by-task results of all V versions, which a leaderboard never sees, so when only V is declared the Bonferroni correction is what is available. On a chain in which every submission is the best of V, the holder is inflated in the same way and the chance of any false certification falls to 0.0% at $V = 10$; the danger is one team trying many versions against a holder that did not. This is why our reporting checklist asks for the number of versions tried: without it the true level of the record test is unknown.

5.2 Do the certification rules keep their promise?

We simulate boards of 40 submissions on 500 tasks with the fitted task difficulties and copula correlation, but with the true score started at 60% rather than the fitted 77%, to leave room for a staircase of improvements (300 boards per scenario; Table 8); the lower base gives more disputed

		Selected-step null		Global null, chain of 40	
V	sets a record	size, selective	size, V-corrected	FWER, selective	FWER, V-corrected
1	48%	3.5%	3.5%	2.5%	2.5%
2	64%	4.9%	2.5%	1.0%	0.0%
3	73%	6.7%	2.1%	0.0%	0.0%
5	81%	8.2%	1.8%	0.0%	0.0%
10	90%	12.4%	1.5%	0.0%	0.0%

Table 5: The record test when the submission is the best of V equally able private tries on the same tasks (fitted Verified model, 500 tasks). Selected-step null: share of submissions that set a record, and how often the selective test and the V-corrected test wrongly reject among them. Global null: share of boards with at least one false certification, on 40-submission chains in which every submission is the best of V.

τ (logit)	Task-level selective test	Repository-level test	SE ratio, repository / task
0.00	3.8%	2.5%	0.88
0.25	9.6%	3.1%	1.01
0.50	24.9%	3.4%	1.32
1.00	49.4%	3.7%	2.08

Table 6: False-rejection rate of the record test when systems have repository-specific strengths (τ on the logit scale): selected-step null on Verified's repositories, 20,000 pairs per level. SE ratio: mean ratio of the repository-level to the task-level standard error of the paired difference; the real board's median for neighbouring top-10 pairs is 1.13.

tasks per step, so the per-step rates here are not those of Table 7, and validity holds in both. Under the global null every record is luck. In the staircase scenario the true score rises by 3 points every 5 submissions, so records are a mix of genuine jumps and lucky steps within a level; in the late scenario one genuine jump of twice that size happens halfway through the chain. A certified step counts as false if the new system's true score is not higher than that of the system it was tested against. Under the global null the unadjusted test certifies a false record on 4.7% of boards, Holm over the finished chain on 3.0%, alpha-spending on 2.7%, online fallback on 2.7%, LORD++ on 1.7%, and the continuity-corrected polyhedral p-value under alpha-spending on 3.3%; all at or below the 5% level. With one late jump of twice the step size, the share of genuine records certified is 14% for Holm, 9% for alpha-spending, 9% for online fallback, 5% for LORD++ and 12% for the polyhedral p-value under alpha-spending, with 0.3%, 1.3%, 1.3%, 1.0% and 2.0% of the certified records being false; the unadjusted test certifies 36% of genuine records, but 5.7% of what it certifies is false. The staircase's jumps of 3 points sit right at the limit of what a 500-task board can see, and no rule with error control certifies more than 1% of them (the unadjusted test certifies 10%); testing every submission that beats the last certified record, instead of only leaderboard records, raises the number of tests to 28.8 per board and the share certified to 11%, with 0.1% of certifications false. Online fallback matches alpha-spending in every scenario here, because genuine records never arrive in runs big enough to see, and by construction it never does worse; it pays off only where several records in a row are genuine and large, which none of the real boards below shows. On the certified baseline, where every submission above the last certified record is tested, the tenure rule of Corollary 3 certifies a false record on 3.0% of global-null boards and test-counting alpha-spending on 4.0%; on the staircase they certify 8% and 9% of genuine steps, and with one late jump 4% and 11%, the price of spending budget on dates with no test. In the variant scenario every submission

Fitted board	Tasks	Steps per board	Global null (holder is the best so far) selective	two-sided	one-sided	Selected-step null (holder not inflated) selective	two-sided	one-sided	Holm, any (selective)	Power at MDD
SWE-bench Verified	300	2.8	0.3%	0.3%	1.4%	3.7%	3.8%	8.2%	0.7%	0.77
	500	2.8	0.5%	0.5%	1.7%	3.7%	3.9%	8.0%	0.7%	0.78
	2,000	3.0	0.8%	0.8%	2.0%	4.4%	4.5%	9.1%	0.7%	0.77
SWE-bench Lite	300	2.9	0.5%	0.5%	1.3%	3.9%	4.0%	8.1%	0.5%	0.76
	500	3.0	0.8%	0.8%	2.1%	4.1%	4.3%	8.3%	1.3%	0.77
	2,000	3.2	0.9%	0.9%	1.8%	4.1%	4.2%	9.2%	1.0%	0.78
Terminal-Bench 2	300	2.6	0.9%	1.0%	1.8%	3.1%	3.4%	7.1%	1.8%	0.74
	500	2.8	0.4%	0.5%	1.7%	3.3%	3.3%	7.5%	0.8%	0.75
	2,000	3.0	0.7%	0.8%	1.9%	4.1%	4.2%	8.5%	1.7%	0.78
LiveBench	300	2.8	1.0%	1.1%	2.2%	3.5%	3.7%	7.9%	2.0%	0.77
	500	2.9	0.7%	0.8%	2.2%	4.0%	4.3%	9.0%	0.8%	0.77
	2,000	3.1	0.9%	1.0%	1.6%	4.4%	4.5%	9.0%	1.5%	0.77

Table 7: How often the record test wrongly rejects among record steps, with the simulation model fitted to each of four boards (their own task difficulties, base score and copula correlation). Global null: 40 equally able systems in sequence, averaged over the chain. Selected-step null: equally able holder and challenger, challenger scored higher. A correct test rejects 5% at the 5% level. Last column: how often the selective test catches a gap equal to the formula's minimum detectable difference, target 0.80.

	Global null			Staircase		One late jump			Variants, global null	
Procedure	FWER	certified	FDR	power	certified	FDR	power	certified	FWER	certified
unadjusted, exact	4.7%	0.05	5.4%	10%	0.7	5.7%	36%	0.42	4.0%	0.04
Holm over the realised chain	3.0%	0.03	0.0%	1%	0.1	0.3%	14%	0.14	1.0%	0.01
alpha-spending, exact	2.7%	0.03	0.3%	0%	0.0	1.3%	9%	0.10	1.3%	0.01
online fallback, exact	2.7%	0.03	0.3%	0%	0.0	1.3%	9%	0.10	1.3%	0.01
LORD++, exact	1.7%	0.02	0.0%	0%	0.0	1.0%	5%	0.06	0.0%	0.00
alpha-spending, polyhedral	3.3%	0.03	0.3%	1%	0.0	2.0%	12%	0.14	3.3%	0.03
LORD++, polyhedral	2.0%	0.02	0.0%	0%	0.0	1.0%	8%	0.09	0.0%	0.00
LORD++, exact, certified baseline	2.3%	0.02	0.1%	11%	2.7	1.0%	8%	0.71	0.7%	0.01
alpha-spending, exact, certified baseline	4.0%	0.04	0.1%	9%	2.4	1.3%	11%	0.81	1.7%	0.02
tenure-indexed spending, exact, certified baseline	3.0%	0.03	0.0%	8%	2.2	1.3%	4%	0.54	0.7%	0.01

Table 8: Certification rules on simulated 40-submission boards of 500 tasks (fitted Verified difficulties and correlation, true score started at 60%). FWER: share of boards with at least one false certification. FDR: expected share of certified records that are false. Power: share of genuine record steps certified. Certified: mean number of certified records per board.

is equally able, but 50% of the submissions after the first are tweaked copies of a random earlier submission (its results with passes flipped to failures at 10% and failures flipped to passes at the rate that keeps the true score the same), the case where the task-level null of Proposition 1 fails even though every record is still luck: the unadjusted test certifies a false record on 4.0% of boards, Holm on 1.0%, alpha-spending on 1.3% and LORD++ on 0.0%, so the rules stay at or below their level there too. Alpha-spending has the stronger guarantee; LORD++ starts with half the budget and earns budget back on each certification, so it does worse when the only genuine record comes early and better on chains with several genuine jumps, as on the real boards below.

5.3 Does the winner's-curse correction work?

We test the corrections on SWE-bench Verified, with Lite and Terminal-Bench as further checks, in four settings. **Known truth.** We treat the real table of results as the whole population, draw a pretend "benchmark" of n tasks from it with replacement, and compare each estimate with the winner's true score on the whole population. This favours a bootstrap method, because the true gaps between systems equal the observed gaps. **Equal leaders.** The hardest case: the top 20 systems are given exactly the same true ability, outcomes are simulated from the fitted task-

Setting	Estimator	Mean error (SE)	Median error	RMSE	Coverage	Length	Defined	Mean, common	Median, common
Known truth, 500 tasks	reported	+0.92 (0.08)	+1.00	1.85	0.94	7.0	100%	+0.95	+1.00
	bootstrap	+0.08 (0.09)	+0.04	1.71	0.95	6.6	100%	+0.12	+0.07
	iterated bootstrap	−0.02 (0.09)	−0.11	1.77	0.94	6.6	100%	+0.03	−0.09
	conditional	+0.05 (0.11)	+0.23	2.10	0.98	9.4	90%	+0.05	+0.23
	hybrid (AKM)	+0.05 (0.11)	+0.23	2.10	0.98	9.5	90%	+0.05	+0.23
	zoom interval (ZF)	+0.92 (0.08)	+1.00	1.85	0.99	10.8	100%	+0.95	+1.00
Equal leaders, 500 tasks	reported	+2.19 (0.08)	+2.15	2.70	0.83	7.3	100%	+2.23	+2.35
	bootstrap	+0.65 (0.08)	+0.67	1.81	0.93	6.6	100%	+0.70	+0.71
	iterated bootstrap	+0.35 (0.09)	+0.32	1.80	0.93	6.6	100%	+0.41	+0.39
	conditional	+0.30 (0.14)	+0.57	2.60	0.97	15.0	86%	+0.30	+0.57
	hybrid (AKM)	+0.30 (0.14)	+0.57	2.60	0.97	10.6	86%	+0.30	+0.57
	zoom interval (ZF)	+2.19 (0.08)	+2.15	2.70	0.99	11.6	100%	+2.23	+2.35
Held-out tasks	reported	+2.04 (0.16)	+2.40	4.06	–	9.9	100%	+2.14	+2.40
	bootstrap	+0.52 (0.16)	+0.63	3.68	–	9.2	100%	+0.66	+0.65
	iterated bootstrap	+0.30 (0.17)	+0.41	3.71	–	9.2	100%	+0.45	+0.42
	conditional	+0.72 (0.20)	+0.94	4.12	–	15.7	82%	+0.72	+0.94
	hybrid (AKM)	+0.72 (0.20)	+0.94	4.12	–	14.2	82%	+0.72	+0.94
	zoom interval (ZF)	+2.04 (0.16)	+2.40	4.06	–	15.7	100%	+2.14	+2.40
Held-out repositories (cluster)	reported	+3.48 (0.17)	+2.98	5.21	–	9.9	100%	+3.90	+3.70
	bootstrap	+1.81 (0.18)	+1.59	4.34	–	15.6	100%	+2.22	+2.13
	iterated bootstrap	+1.85 (0.18)	+1.97	4.48	–	15.6	100%	+2.24	+2.37
	conditional	+3.38 (0.24)	+3.71	5.90	–	13.1	82%	+3.38	+3.71
	hybrid (AKM)	+3.43 (0.23)	+3.71	5.82	–	12.2	82%	+3.43	+3.71
	zoom interval (ZF)	+3.48 (0.17)	+2.98	5.21	–	13.5	100%	+3.90	+3.70

Table 9: Winner's-curse correction on SWE-bench Verified, at least 400 repetitions per setting. Error is estimate minus target in percentage points; positive means overstated. Coverage is shown only where the truth is known; length is the median length of the 95% interval in points. The zoom interval's point estimate is the reported score. The last two columns use the repetitions in which all six estimators are defined.

difficulty model, and the truth is known. **Held-out tasks.** The real 500 tasks are split at random into two halves; the winner is picked on the first half and the estimate is compared with its score on the second half, which is a fair but noisy stand-in for the truth. **Held-out repositories.** The same, but the halves are made of different repositories, chosen among the 504 repository subsets that put 40 to 60% of the tasks in the selection half (200 to 300 tasks). The selection board keeps its repository labels and the corrections resample repositories. The validation score is adjusted for the average difference in difficulty between the halves across all systems; the supplement (Table S4) reports the unadjusted target and the task-resampling corrections as well. Alongside our two corrections we run the hybrid and zoom baselines of Section 3. Table 9 gives the mean error with its simulation standard error, the median error, the root mean squared error, how often the 95% interval contains the truth (its coverage) and its median length where the truth is known, how often each method is defined at all, and the mean and median error on the subset of repetitions in which all six methods are defined, since the conditional and hybrid methods break down exactly where selection matters most, at an exact tie for first.

Results. In the known-truth runs the winner's reported score is too high by 0.9 points at 500 tasks, 1.7 at 250 and 3.6 at 100; with equally able leaders by 2.2 at 500 tasks and 3.0 at 250; and on real held-out tasks by 2.0 points. After the iterated bootstrap correction 0.0 points remain at 500 tasks (standard error 0.09), +0.3 with equal leaders and +0.3 on held-out tasks, so it removes about 84% and 85% of the inflation in the last two cases; the plain bootstrap removes 74% on held-out tasks. These percentages carry simulation errors of about ±8 points, and the held-out repetitions are different splits of one board, so they are not independent. When the held-out half is made of other repositories, the repository-resampling iterated correction removes 47%, so a benchmark whose tasks drift away from the ones the winner was picked on will still overstate its winner. The correction removes the inflation but does not make the winner's score more precise: on held-out tasks the root mean squared error of the iterated bootstrap (3.7) is not below that

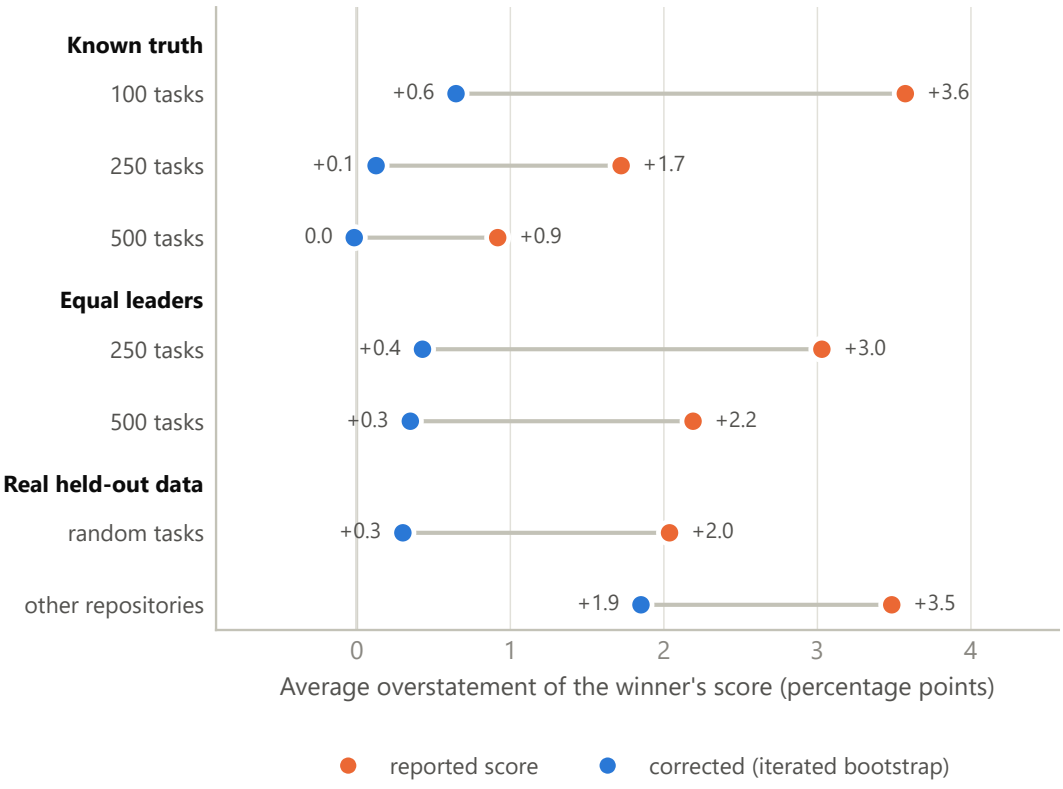


Figure 2: Average overstatement of the winner's score before and after the iterated bootstrap correction, on SWE-bench Verified. Orange: the reported score. Blue: the corrected score. The top two groups have known truth; the bottom group uses real held-out data, where the target is the winner's score on tasks that were not used to pick it.

of the plain bootstrap (3.7), because most of the error in one system's score is ordinary sampling noise, not selection. On the common subset the conditional method, which is median-unbiased by construction, has a median error of +0.6 with equal leaders against +0.4 for the iterated bootstrap, but it is defined in only 86% of those runs. The hybrid interval fails only at an exact tie for first and never runs off to infinity: with equal leaders it contains the truth in 0.97 of runs with a median length of 10.6 points, against 0.97 and 15.0 for the conditional interval, 0.93 and 6.6 for the iterated bootstrap, and 0.99 and 11.6 for the zoom interval, which leaves the point estimate as reported; the hybrid estimate's median error on the common subset is +0.6 points. The hybrid estimate came out identical to the conditional one in every setting we ran, because the wide first-stage interval seldom binds at these sample sizes; its gain over the conditional method is the shorter interval, and on held-out tasks it leaves +0.7 points of inflation against +0.3 for the iterated bootstrap. The two baselines do not change the recommendation: the iterated bootstrap has the smallest bias among estimates that still work at a tie, and the hybrid interval is the one to report when a guarantee under normality is wanted. In the known-truth runs the 95% iterated interval contains the truth 0.90 of the time at 100 tasks, against 0.83 for the interval around the reported score, and 0.94 against 0.94 at 500; it stays below the promised 95% when tasks are few. On Lite the iterated correction removes 74% on held-out tasks but overshoots on held-out repositories (mean error −0.6 points against 2.2 for the reported score, with selection halves of only 120 to 180 tasks).

Tasks	Known truth: interval covers true rank			Equal leaders: interval covers			Equal leaders: confidence set		
	all	true top 10	selected winner	rank 1 (min)	average rank	tied block	all	contains each leader	median size
250	0.96	0.95	0.96	0.44	0.95	1.00	0.88	0.99	28
500	0.96	0.96	0.97	0.42	0.95	1.00	0.88	0.99	23

Table 10: How often the 95% rank intervals and the set of possible best systems contain the truth, SWE-bench Verified, known-truth simulation. Equal leaders: the 20 top systems have equal true ability; the three conventions treat their true rank as 1, as the average of the tied ranks, or as the whole tied block.

On Terminal-Bench (Supplement, Table S5) the known-truth and equal-leaders results point the same way, but on real held-out halves of 44 tasks the iterated correction overshoots (mean error -1.2 points against 2.0 for the reported score), so the correction is not guaranteed to help on very small benchmarks. On LiveBench (Supplement, Table S5) equally able leaders are inflated by 3.9 points, of which the iterated bootstrap leaves +0.5 and the hybrid 0.0; on held-out halves of its questions there is no inflation to remove (-0.5 for the reported score), because its leader is far ahead of the field, and the corrections leave it nearly untouched (-0.6 and -0.5).

5.4 Do the rank intervals and the set of possible best systems contain the truth?

We use the known-truth setting again (300 repetitions; Table 10). The 95% rank interval contains the true rank for 0.96 (250 tasks) and 0.96 (500 tasks) of all systems, for 0.95 and 0.96 of the ten truly best systems, and for 0.96 and 0.97 of the winners. When the top 20 systems are equally able, a leader's "true rank" is not well defined: if tied systems all count as rank 1 the interval contains it for only 0.44 and 0.42 of the leaders, if they count as the average of the tied ranks for 0.95 and 0.95, and it overlaps the block of tied ranks in 1.00 and 1.00 of cases. The intervals show how much a rank moves when the tasks are redrawn and should not be read as confidence intervals for a true rank when leaders are close [36, 38]. The set of possible best systems behaves differently: in the same runs it contained each of the equally able leaders in 0.99 (250 tasks) and 0.99 (500 tasks) of repetitions, with a median size of 28 and 23 against 20 tied systems, so it keeps each tied leader at the promised rate without being much larger than the tied block. The supplement (Table S3) shows how often the observed winner really is one of the best systems in the known-truth runs: 0.70 at 500 tasks, 0.65 at 250 and 0.39 at 100.

6 Audit of six agent leaderboards, with a non-agent comparison

Records. Only 20 of 74 record steps (27%) are clearly better than the record they replaced by the selective test, and 13 (18%) after Holm; the two-sided test finds the same 20 steps, and the one-sided test at 5%, which wrongly rejects about 8.0% of lucky steps in simulation, finds 24. The split by benchmark size is stark: 9 of 13 on SWE-bench Test, 11 of 61 on the five boards with at most 510 tasks. Among the 35 steps that won by less than two points, 0 pass the test. Table 14 shows how the count moves with the choices we made: the repository-level test, the cautious one where tasks share repositories (Section 5), leaves 10 of the 65 SWE-bench steps against 19 with the task-level test, and ordering same-date submissions worst first creates more steps (99) of which 29% pass, so the share of records we cannot confirm barely changes. A record that does not pass is not shown to be equal to the one before it: for those records the margin could plausibly be as large as 4.6 points on Verified (the median 97.5% upper bound; 7.2 on Lite, 9.9 on Terminal-Bench), so gains of that size are neither shown nor ruled out. These are single-run submissions.

Board	Systems	Tasks	Top-rank freq.	95% set	Records sig.	Holm	Repository-level	MDD (pts)	Leader (%): reported → adjusted
SWE-bench Verified	135	500	0.35	14	6/30	2	1/30	4.3	79.2 → 78.1
SWE-bench Lite	84	300	0.47	5	3/15	1	1/15	6.7	60.3 → 58.7
SWE-bench Test	24	2,294	1.00	1	9/13	9	7/13	2.2	52.6 → 52.6
SWE-bench Multimodal	12	510	0.38	5	1/7	0	1/7	3.8	36.5 → 35.3
SWE-bench Multilingual	14	300	0.54	9	0/0	0	0/0	7.2	72.7 → 71.2
Terminal-Bench 2	67	89	0.80	10	1/9	1	–	7.9	89.9 → 89.0
All boards					20/74	13	10/65		

Table 11: Audit summary. Top-rank freq.: share of redraws of the tasks in which the leader has the top score. 95% set: number of systems that could be the true best. Records sig.: record steps clearly better than the previous record at the 5% level (selective exact paired test, tasks as the unit) over all record steps. Holm: still clearly better after the Holm correction within the board. Repository-level: the same count with repositories as the unit, the cautious one. MDD: smallest true gap the board catches with 80% power. Last column: the leader's reported score and its winner's-curse adjustment (iterated bootstrap); where two systems tie for first, the leader is the one that ranks first more often.

Certified records. Run in date order, the two guaranteed rules certify fewer records than Holm does looking back, and LORD++ as many (Table 13): alpha-spending 11 and online fallback 11, the two rules with the proved guarantee, and LORD++, checked by simulation, 13 of the 74 leaderboard records, against 13 for Holm and 20 with no correction at all; the polyhedral p-value, which also allows for the holder's selection, gives 13 under LORD++. On these chains no two records in a row are both genuine and large enough to see, so the fallback rule certifies the same 11 records as spending, and the same 2 of the 4 LiveBench records (Table 16). On each board, with 95% confidence, every record the rules certify is a real improvement on the record it replaced (a rejection says the old record does not dominate the new one task by task); the guarantee is per board, not over the six boards at once. With the certified baseline (Corollary 3), every submission that beats the last certified record is tested: the tenure rule, which carries the whole-chain guarantee, certifies 20 of 160 tested submissions across the boards, test-counting alpha-spending 24 of 137, and LORD++ 27 of 124. Figure 3 shows the leaderboard record line and the certified record line under the tenure rule on Verified and Lite: on Verified the certified leader changes 6 times in 135 submissions, against 30 leaderboard records.

The rule at work, and the records that came with a write-up. Table 12 lists all 30 Verified records in date order: the system's name as the leaderboard shows it, its score, its margin, the per-date factor g, the selective p-value, the level the online fallback rule uses at that step and its verdict, and what the entry links as its write-up: a paper or technical report, a blog or news post, or only a code or product page. Run live, the rule starts at a level of 3.0% (the first record's share of the 5% budget), certifies the first record at once ($p < 0.001$), and passes its level on, so the second record is tested at a higher level than spending alone would give; that record fails, the level falls back to the spending share, the third record ($p = 0.001$) is certified, and after that every p-value sits above the shrinking level. It certifies 2 of the 30 records; the other 4 that pass the per-step test at 5% have p-values between 0.02 and 0.05, which is not enough once the budget has been spent on earlier records. 14 of the 30 records link a write-up as their report, 2 of them a paper or technical report and the rest a blog or news post, and only 1 of the 14 is clearly better than the record it replaced; 3 records carry the maintainers' "checked" tag, and 1 of those is. None of this says those write-ups are wrong: a report usually documents a system, not a claim about this one gap, and a record the board cannot confirm may still be real. It says that on this board the published record and the certified record are different things, and that a maintainer running the rule would have

#	Date	System, as named on the leaderboard	Score	Margin	g	p_{sel}	Level	Certified	Write-up
1	2024-04-02	SWE-agent + GPT 4 (1106)	22.4	+18.0	4	<0.001	0.0304	yes	site
2	2024-05-09	Amazon Q Developer Agent (v20240430-dev)	25.6	+3.2	1	0.157	0.0380	no	site
3	2024-06-12	MASAI + GPT 4o (2024-06-12)	32.6	+7.0	1	<0.001	0.0034	yes	site
4	2024-06-17	Factory Code Droid	37.0	+4.4	1	0.046	0.0053	no	site
5	2024-06-28	AutoCodeRover (v20240620) + GPT 4o (2024-05…	38.4	+1.4	1	0.558	0.0012	no	site
6	2024-07-21	Amazon Q Developer Agent (v20240719-dev)	38.8	+0.4	1	1.000	8.4×10^{-4}	no	site
7	2024-08-20	Honeycomb	40.6	+1.8	2	0.870	6.2×10^{-4}	no	site
8	2024-08-24	Gru(2024-08-24)	45.2	+4.6	1	0.043	4.7×10^{-4}	no	site
9	2024-09-24	Solver (2024-09-12)	45.4	+0.2	1	1.000	3.8×10^{-4}	no	site
10	2024-10-22	Tools + Claude 3.5 Sonnet (2024-10-22)	49.0	+3.6	2	0.186	3.0×10^{-4}	no	site
11	2024-10-28	Solver (2024-10-28)	50.0	+1.0	2	1.000	2.5×10^{-4}	no	site
12	2024-10-29	OpenHands + CodeAct v2.1 (claude-3-5-sonnet…	53.0	+3.0	2	0.301	2.1×10^{-4}	no	post
13	2024-11-08	devlo	54.2	+1.2	2	1.000	1.8×10^{-4}	no	post
14	2024-12-02	Amazon Q Developer Agent (v20241202-dev)	55.0	+0.8	2	1.000	1.6×10^{-4}	no	site
15	2024-12-08	Gru(2024-12-08)	57.0	+2.0	1	0.395	1.4×10^{-4}	no	site
16	2024-12-13	devlo	58.2	+1.2	1	0.641	1.2×10^{-4}	no	post
17	2024-12-21	CodeStory Midwit Agent + swe-search	62.2	+4.0	1	0.049	1.1×10^{-4}	no	site
18	2025-01-10	Blackbox AI Agent	62.8	+0.6	2	1.000	9.4×10^{-5}	no	site
19	2025-01-17	W&B Programmer O1 crosscheck5	64.6	+1.8	1	0.422	8.4×10^{-5}	no	post
20	2025-03-16	Augment Agent v0	65.4	+0.8	1	0.819	7.6×10^{-5}	no	post
21	2025-04-15	OpenHands	65.8	+0.4	1	1.000	6.9×10^{-5}	no	post
22	2025-04-30	Zencoder (2025-04-30)	70.0	+4.2	1	0.024	6.3×10^{-5}	no	post
23	2025-05-15	Refact.ai Agent	70.4	+0.4	1	1.000	5.7×10^{-5}	no	post
24	2025-05-19	TRAE	70.6	+0.2	2	1.000	5.3×10^{-5}	no	post
25	2025-05-22	Tools + Claude 4 Opus (2025-05-22)	73.2	+2.6	3	0.496	4.9×10^{-5}	no	post
26	2025-06-03	Refact.ai Agent + Claude 4 Sonnet + o4-mini	74.4	+1.2	1	0.608	4.5×10^{-5}	no	post
27	2025-06-12	TRAE + Claude Sonnet 4 + Opus 4 + Sonnet 3.…	75.2	+0.8	1	0.771	4.2×10^{-5}	no	post
28	2025-08-04	EPAM AI/Run Developer Agent v20250719 + Cla…	76.8	+1.6	2	0.726	3.9×10^{-5}	no	site
29	2025-09-28	TRAE + Doubao-Seed-Code	78.8	+2.0	1	0.289	3.6×10^{-5}	no	paper
30	2025-12-05	Sonar Foundation Agent + Claude 4.5 Opus	79.2	+0.4	1	1.000	3.4×10^{-5}	no	paper

Table 12: Every record on SWE-bench Verified, in date order, with the online fallback rule run live. Score: the challenger's reported score (%). Margin: its gain over the record it replaced (points). g: submissions sharing the challenger's date (the per-date factor). p_{sel}: the selective p-value after that factor. Level: the level the fallback rule uses at that step; certified when p_{sel} is at or below it. Write-up: what the submission's leaderboard entry links as its report (paper: an arXiv paper or technical report; post: a blog or news post), or, with no report link, only a code or product page (site).

announced 2 new states of the art over the board's history rather than 30.

Resolution. Figure 4 compares the smallest gap each board can reliably see with the gaps records are claimed by. SWE-bench Verified catches true gaps of 4.3 points with 80% power (95% interval from redrawing tasks 3.8 to 4.7; 4.8 with repository-level variance), calls a typical margin significant from about 3.0 points, and its typical record wins by 1.5; seeing a one-point gap would take about 9,300 tasks (interval 7,100 to 10,800) and a two-point gap about 2,300. Lite can see 6.7 points against a typical record margin of 3.0, Multimodal 3.8 against 1.2, Terminal-Bench 2 7.9 against 4.0 with 89 tasks, and SWE-bench Test 2.2 against 3.3, the one board whose typical record is bigger than the smallest gap it can see. Table 15 gives how far back a record has to be compared before it is clearly better; the supplement draws the ladders themselves (Figure S1).

Ranks. Figure 5 shows the leaders. On Verified the two tied leaders rank first in 0.35 and 0.32 of redraws of the tasks, and 14 systems could be the true best (20 with the all-pairs bound; 21 when repositories are redrawn). Terminal-Bench 2 shows the opposite case: the leader is first in 0.80 of redraws, yet with 89 tasks 10 systems still cannot be ruled out, because systems well below the leader cannot be excluded with so few tasks. A clear leader and a long list of possible best systems can both be true: the set keeps each truly best system at about the promised rate but cannot exclude systems well below the leader with so few tasks, so its size overstates how many

Board	Records	Unadjusted	Holm	Spending	Fallback	LORD++	LORD++ (polyhedral)	Spending, certified baseline	Tenure, certified baseline
SWE-bench Verified	30	6	2	2	2	2	2	7/56	6/71
SWE-bench Lite	15	3	1	1	1	1	1	4/31	4/31
SWE-bench Test	13	9	9	7	7	9	9	9/13	7/13
SWE-bench Multimodal	7	1	0	0	0	0	0	2/8	2/8
SWE-bench Multilingual	0	0	0	0	0	0	0	0/0	0/0
Terminal-Bench 2	9	1	1	1	1	1	1	2/29	1/37
All boards	74	20	13	11	11	13	13	24/137	20/160

Table 13: Records certified by each rule, run in date order with the exact selective p-value unless noted. Unadjusted: selective exact test at 5% per step. Holm: looking back over the finished chain. Spending and fallback (both with the proved whole-chain guarantee) and LORD++ (checked by simulation): live levels. Certified baseline: every submission that beats the last certified record is tested (shown as certified/tested); the tenure levels carry the whole-chain guarantee of Corollary 3, the test-counting spending levels are checked by simulation.

Board	Steps	Selective (primary)	Two-sided	One-sided	Repository-level	Worst-first ordering
SWE-bench Verified	30	6	6	6	1	9/36
SWE-bench Lite	15	3	3	6	1	4/20
SWE-bench Test	13	9	9	10	7	13/17
SWE-bench Multimodal	7	1	1	1	1	1/7
SWE-bench Multilingual	0	0	0	0	0	1/10
Terminal-Bench 2	9	1	1	1	–	1/9
All boards	74	20	20	24	10 (of 65)	29/99

Table 14: Number of record steps that pass at the 5% level under five choices: the primary selective exact test with best-first ordering of same-date submissions, the two-sided exact test, the one-sided test at 5%, the repository-level sign-flip test (the cautious count), and worst-first ordering (more steps, shown as passing/steps).

systems could really be best.

Adjusted winners. The winner's-curse adjustment of the leader's score, which covers luck in which tasks were drawn and nothing else, is in the last column of Table 11. On Verified the leader's reported 79.2% becomes 78.1% (95% interval 74.9 to 81.5); on Lite 60.3% becomes 58.7% (53.4 to 63.7). On SWE-bench Test the adjustment is tiny, because the leader is far ahead and the benchmark is large. Tuning on public tasks is not covered, so the true inflation may be larger. The conditional method breaks down on Verified and Multimodal, where the top is an exact tie, and gives 55.7% on Lite and 89.9% on Terminal-Bench.

Run randomness on Terminal-Bench. Terminal-Bench runs each task several times, which lets us split the noise in a score into two parts: which tasks were drawn, and the agent giving different answers from one run to the next. For the 20 strongest systems, run-to-run randomness is a median 36% (range 14 to 64%) of the per-task variance of a single-run score, and 10% of a five-run score; for the difference between neighbouring top-10 systems it is 62% with one run and 20% with five. So run noise is a large part of the noise in a single-run coding-agent score, about a third of its variance and 62% of the variance of a single-run difference between neighbouring systems, something the SWE-bench boards, with one run per submission, cannot show.

Development selection, seen through repeated runs. How much teams gain by privately trying several versions before submitting cannot be measured from one run per system, but the Terminal-Bench submissions that ran the whole benchmark several times show the scale of its

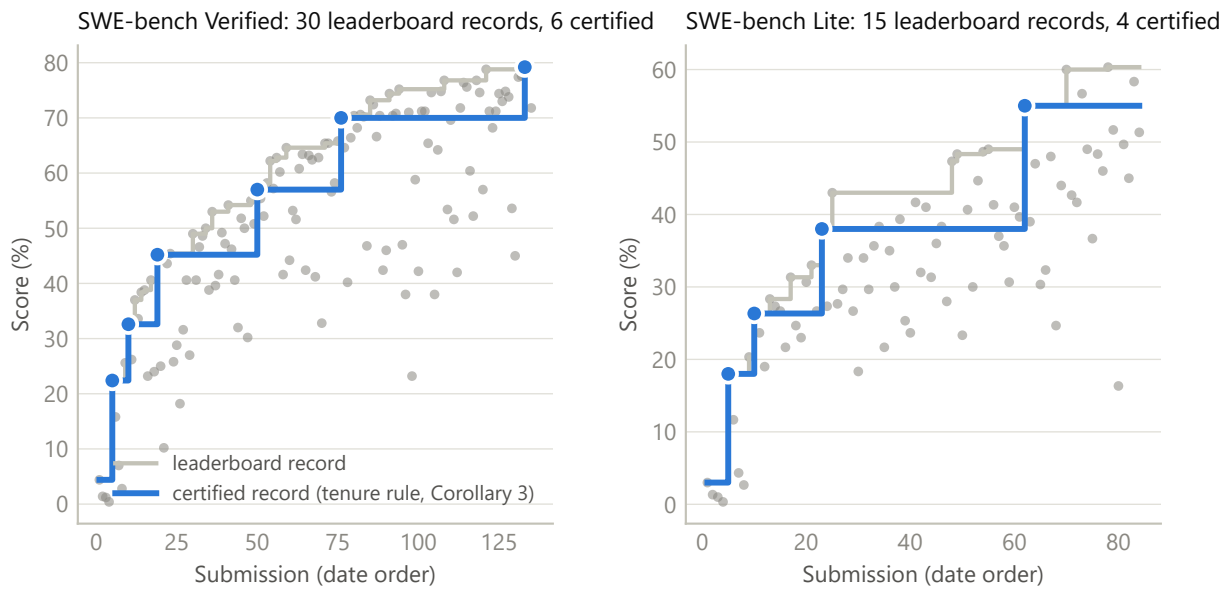


Figure 3: Leaderboard records and certified records in submission order. Gray dots: all submissions. Gray step: the leaderboard record. Blue step and dots: the certified record under the tenure rule of Corollary 3, which changes only when a submission is certified against the last certified record.

Board	Median margin (pts)	Median lag	Beats previous record	Beats no earlier record
SWE-bench Verified	1.5	3	20%	0%
SWE-bench Lite	3.0	2	20%	0%
SWE-bench Test	3.3	1	69%	0%
SWE-bench Multimodal	1.2	2	14%	14%
Terminal-Bench 2	4.0	3	11%	0%

Table 15: Lag analysis. Typical (median) margin of a record; median number of records back one must go before a record is clearly better than that earlier record (lag); the share of records clearly better than the one they replaced; and the share clearly better than no earlier record on the board.

simplest form. 24 kept submissions have at least two complete jobs (median 5); the spread (standard deviation) of a submission's score across its jobs has median 1.7 points (middle half of submissions 1.0 to 2.5), and a team that had reported its best job instead of the average of its jobs would have gained a median 1.9 points (mean 2.7, at most 12.9). As a rough guide to scale, consider a team that tries V versions whose scores scatter like repeated runs and submits the best: the expected gain is $\mathbb{E}[\max_{v \le V} Z_v]$ times that spread, 1.0 points for two versions, 2.0 for five and 2.7 for ten, of the same order as the winner's-curse inflation from task sampling, and below this board's typical record margin (2.0 against 4.0 points). The leaderboard reports the average over jobs, so this is a bound on what reporting the best run would add, not a finding that anyone did it, and picking among versions that differ by more than run noise can add more.

A non-agent comparison. The procedures are not special to agents, and LiveBench shows what they return on a leaderboard of language models scored automatically question by question (Table 16). With 318 questions it can see gaps no finer than Lite can (minimum detectable difference 6.5 points, about 13,300 questions to see one point). Its 4 record steps under our main ordering

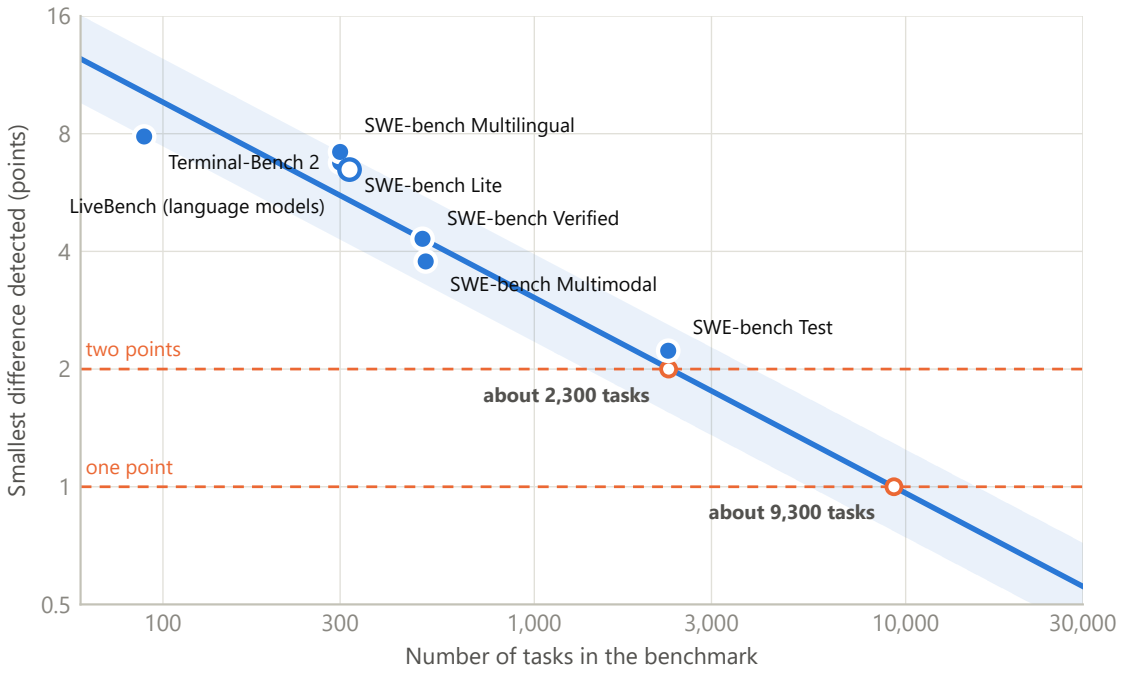


Figure 4: What a point costs. The smallest true gap a benchmark can catch with 80% power in the two-sided paired test at the 5% level, against the number of tasks (both axes logarithmic). The line uses SWE-bench Verified's task-by-task spread and the band spans the boards; orange circles mark the number of tasks Verified would need to see one- and two-point gaps. Filled dots are the six agent boards; the hollow dot is LiveBench, the language-model comparison.

win by a typical 5.8 points, above the board's critical margin of 4.5; 4 pass the unadjusted test, 1 Holm, and the live rules certify 2 (alpha-spending), 2 (online fallback) and 0 (LORD++), the last of those certifications within the Monte Carlo error of the sign-flip test ($p = 0.0018$ against a level of 0.0019), so that count is fragile; under the worst-first ordering of the 8 models that share the first date there are 11 steps, 6 of them clearly better. Its leader ranks first in 1.00 of redraws and is the only system that could be best, so the winner's-curse correction leaves its score where it is (77.6% to 77.7%). Four records are too few to compare with the agent boards, and the board is a reconstruction (three of six categories, partial credit, models dated by their first judgment in a snapshot file); what the comparison shows is that the tools apply unchanged, and that on a board that can see the same gaps it is the size of the margins that moves the counts.

7 The honestboard package

`honestboard` (it needs only numpy and scipy) implements everything above. A `Board` holds the systems-by-tasks table, with loaders for JSON, CSV and a checkout of the SWE-bench `experiments` repository. `Board.check_claim(system)` returns, in one call, the observed rank and its interval, how often the system ranks first, whether it could be the best, the paired test against its closest rival, and, for the leader, the winner's-curse-corrected score. `Board.record_audit()` walks through a leaderboard's history and tests every record step; `Board.resolution()` gives the smallest gap the board can see and the tasks needed for a one-point gap. A command line (`python -m honestboard report BOARD.json`) prints the same, and `pr-check` reads a checkout of a leaderboard repository

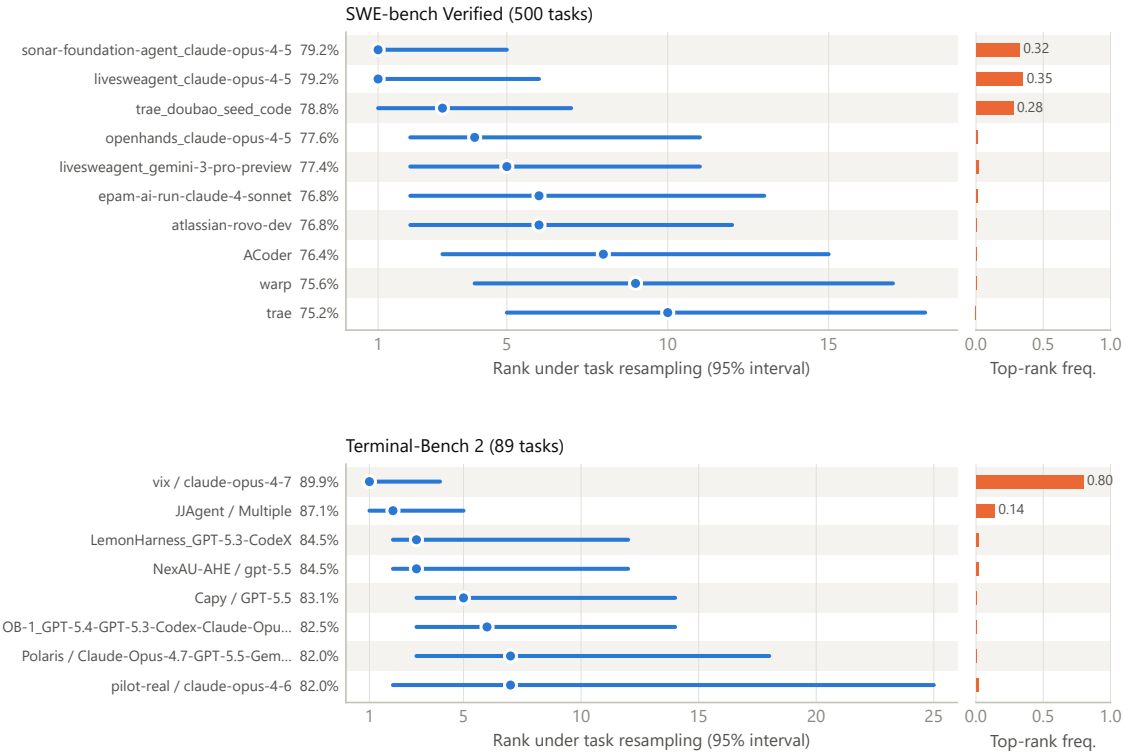


Figure 5: Rank intervals (95%, from redrawing the tasks) for the leading systems. The dot is the observed rank and the reported score follows each name. Orange bars: share of redraws in which the system ranks first.

and prints a Markdown report on one new submission, so that a maintainer can post it on every pull request. The package has unit tests that include simulations checking that the set of possible best systems and the conditional method contain the truth as often as promised, brute-force checks of the cut-off limits and of the sign-flip enumeration, tie handling, input validation and the covariance estimator.

8 Limitations

- **Task sampling only.** We model luck in which tasks were chosen (Table 2). Tuning on the public tasks is not modelled, and it is where much benchmark inflation comes from [44, 52]; nothing here says how much of a leaderboard's progress is tuning; the best-of-runs bound from Terminal-Bench covers only the case where the versions tried differ by run noise, and the private-tries simulation prices the simplest form of tuning (Table 5) but not training on the tasks. Run-to-run randomness is not modelled for SWE-bench, where submissions are single runs; on Terminal-Bench it is about a third of the variance of a single-run score (14 to 64% across systems) and 62% of the variance of a single-run difference between neighbouring systems.
- **What the numbers are about.** Our statements are about more tasks of the benchmark's own kind, not about unseen repositories or real-world use. The held-out-repository results show how quickly the correction gets worse when the kind of task changes.

	LiveBench	SWE-bench Verified	Terminal-Bench 2
Type of system	language model	agent	agent
Per-task score	credit in $[0, 1]$	pass or fail	mean of 1 to 10 trials
Systems	94	135	67
Tasks	318	500	89
Record steps	4	30	9
Distinguishable (selective exact test)	4	6	1
After Holm	1	2	1
Certified, LORD++	0	2	1
Certified, alpha-spending	2	2	1
Certified, online fallback	2	2	1
Median record margin (pts)	5.8	1.5	4.0
Minimum detectable difference (pts)	6.5	4.3	7.9
Tasks to resolve one point	13,300	9,300	5,500
Leader's top-rank frequency	1.00	0.35	0.80
Systems tied at the top	1	2	1
95% confidence set of the best	1	14	10
Leader's score, reported $\rightarrow$ corrected (%)	77.6 $\rightarrow$ 77.7	79.2 $\rightarrow$ 78.1	89.9 $\rightarrow$ 89.0

Table 16: The non-agent comparison. LiveBench (language models) beside the agent board with the longest record chain and the smallest agent board. Certified counts are from the live rules run in date order on the leaderboard baseline.

- **Tasks are not independent.** Tasks from one repository move together. The task-level tests and intervals treat tasks as independent, which makes things look more certain than they are: with repository-specific strengths of the size the real board's standard errors suggest, the task-level record test wrongly rejects 10 to 25% of lucky record steps (Table 6). The repository-level test holds its level but has little power with a dozen repositories (Multilingual has 41), and its standard error comes out too small when one repository dominates; we report both counts and call the repository-level one the cautious one.

- **Record test and certification.** Record steps are selected for being records. The selective test wrongly rejects at most 5% under both nulls, at every chain position we simulated and on split halves of the real boards; it is exact for a fixed holder and cautious for a holder that is the best so far, under the task-level null (Proposition 1), which asks more than equal true scores and assumes outcomes independent across tasks; the case where that null fails (tweaked copies of earlier systems), the certified baseline with test-counting levels (no proof, per step or over the chain), and the polyhedral variant rest on simulation. The alpha-spending guarantee is exact given per-step validity but loses power late in a chain; LORD++'s guarantee assumes independent p-values, ours are linked through shared tasks, and we rely on simulation for it.

- **No theory for the correction.** The iterated bootstrap is justified by simulation only; the conditional method is exact only under normality and breaks down at ties; the correction overshoots on the smallest benchmark; its intervals contain the truth less often than promised at small sizes. The hybrid and zoom baselines carry coverage guarantees under normality, and Section 5 reports how they compare.

- **The comparison board.** LiveBench's judgments file covers three of its six categories, its questions change over time, and we date a model by its first judgment, so its record chain is only an approximation of the leaderboard's history; its per-question scores are partial credit rather than pass or fail.

- **Rank intervals.** They show how much a rank moves when the tasks are redrawn and do not contain a true rank when leaders are tied; use the set of possible best systems for that question.
- **Same-date ordering.** Which steps count as records depends on how same-date submissions are ordered; we report both extremes and prefer the one that invents no record. The best-first challenger is the best of its date, which the per-date factor of Section 3 corrects for; the proofs assume that, apart from this, which systems were submitted and in what order does not depend on their results, which a submission time finer than the day would make true without any correction.
- **Missing task-by-task data.** 47 of 182 Verified submissions have no task-by-task file. They are recent and score high; by overall score none would have held the record, but they are missing from every other analysis of that board, and other leaderboards may have similar gaps.
- **Terminal-Bench handling.** Most job files do not say which benchmark version they ran, pooled submissions have uneven run counts, and runs that errored are counted as failures; other choices would move its numbers.
- **No cross-benchmark test of the correction.** Few systems appear on two SWE-bench splits, they are old, and they are spread so widely that the winner is far ahead of the field, so there is no winner's curse to detect.

9 Recommendations

For benchmark maintainers: publish task-by-task results for every submission; call a record a new state of the art only when it is certified against the previous record by the selective record test, at a level set by the online fallback rule (or alpha-spending), whose guarantee needs no independence between steps, so that the error rate holds over the whole history, in the spirit of the Ladder mechanism [4]; LORD++ earns back more budget but is checked only by simulation here. On a board of a few hundred tasks this means that most records of a point or two will never be certified; that is the honest state of affairs, and the argument for growing the benchmark. Report the smallest gap the benchmark can see and the number of tasks needed to see a one-point gap, and expand or retire benchmarks that cannot see the gaps their leaderboards display. For authors: report paired intervals against the strongest baseline, say how many versions were tried on the benchmark so that the record test can be corrected for it (Section 5), and when claiming the top spot report the reported score, the winner's-curse-adjusted score and its interval together, with the hybrid interval where a coverage guarantee under normality is wanted, and say that tuning on public tasks is not covered. For readers: treat a margin below the board's critical margin (about 3.0 points on Verified) as unresolved, not as a tie. Appendix A lists ten things a leaderboard can adopt.

10 Conclusion

Leaderboards compress a large table of task-by-task results into one ranking. On the four agent boards with at most 510 tasks and at least one record, most records cannot be told apart from the record before them by a test that allows for how records are selected; on the 2,294-task SWE-bench Test most can. This is a statement about what the benchmarks can see, not a finding

that the records are equal, and tuning on public tasks, which we do not model, is a separate source of inflation. The procedures work unchanged on a language-model leaderboard, LiveBench, whose four records are too few to compare with the agent boards. Task-by-task data, which several leaderboards already publish, is enough to say how much a record claim shows, and the live certification rules let a leaderboard say it at the moment the claim is made.

Use of AI assistance. The code, the analyses and the drafting of this paper were carried out with an AI assistant (Claude, Anthropic) under the authors' direction; the authors specified the questions, the data and the methods, reviewed every result and every proof, and are responsible for the content. Three outside automated reviews were used to find errors before submission; their findings and the responses are recorded in the repository.

Reproducibility. The script `make_numbers.py` generates every number in this paper from the result files, so the text cannot disagree with the analysis. Data provenance, the command behind every table and figure, the pinned environment and the hashes of every data and result file are in the repository (`https://github.com/jopoku16/honestboard`, file `REPRODUCE.md`).

Supplementary material. Tables S1 to S5 and Figure S1 (the record test by position in the chain, its power, the ranking lottery, the full winner's-curse validation tables for Verified, Terminal-Bench and LiveBench, and the record ladders of five boards) are in the supplement that accompanies the paper (`supplement.pdf` in the repository), generated by the same script as the main text.

References

[1] Isaiah Andrews, Toru Kitagawa, and Adam McCloskey. Inference on winners. *The Quarterly Journal of Economics*, 139(1):305–358, 2024. doi: 10.1093/qje/qjad043.

[2] Mohsen Arjmandi. Harness or model? isolating the harness effect in agentic coding with a contamination-controlled private suite, 2026. arXiv:2609.11987.

[3] Soham Bakshi, Lingjun Gao, Zijun Gao, and Snigdha Panigrahi. Flexible inference for winners with conditional validity, 2026. arXiv:2607.18545.

[4] Avrim Blum and Moritz Hardt. The ladder: A reliable leaderboard for machine learning competitions. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1006–1014, 2015.

[5] Sam Bowyer, Laurence Aitchison, and Desi R. Ivanova. Position: Don't use the CLT in LLM evals with fewer than a few hundred datapoints. In *International Conference on Machine Learning*, 2025. arXiv:2503.01747.

[6] Dallas Card, Peter Henderson, Urvashi Khandelwal, Robin Jia, Kyle Mahowald, and Dan Jurafsky. With little power comes great responsibility. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020. arXiv:2010.06595.

[7] Ivi Chatzi, Eleni Straitouri, Suhas Thejaswi, and Manuel Gomez-Rodriguez. Prediction-powered ranking of large language models. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2402.17826.

[8] Wei-Lin Chiang et al. Chatbot arena: An open platform for evaluating LLMs by human preference, 2024. arXiv:2403.04132.

[9] Bradley Efron. Increasing properties of Pólya frequency functions. *The Annals of Mathematical Statistics*, 36(1):272–279, 1965. doi: 10.1214/aoms/1177700288.

[10] Bradley Efron. Tweedie's formula and selection bias. *Journal of the American Statistical Association*, 106(496):1602–1614, 2011.

[11] Bradley Efron. Estimation and accuracy after model selection. *Journal of the American Statistical Association*, 109(507):991–1007, 2014.

[12] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall, 1993.

[13] William Fithian, Dennis Sun, and Jonathan Taylor. Optimal inference after model selection, 2014. arXiv:1410.2597.

[14] Dean P. Foster and Robert A. Stine. α-investing: A procedure for sequential control of expected false discoveries. *Journal of the Royal Statistical Society: Series B*, 70(2):429–444, 2008. doi: 10.1111/j.1467-9868.2007.00643.x.

[15] Peter R. Hansen, Asger Lunde, and James M. Nason. The model confidence set. *Econometrica*, 79(2):453–497, 2011.

[16] David Heineman, Valentin Hofmann, Ian Magnusson, Yuling Gu, Noah A. Smith, Hannaneh Hajishirzi, Kyle Lo, and Jesse Dodge. Signal and noise: A framework for reducing uncertainty in language model evaluation, 2025. arXiv:2508.13144.

[17] Peter Hoff and Surya Tokdar. Selective and marginal selective inference for exceptional groups, 2025. arXiv:2509.13538.

[18] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[19] Jason C. Hsu. Simultaneous confidence intervals for all distances from the "best". *The Annals of Statistics*, 9(5):1026–1034, 1981. doi: 10.1214/aos/1176345582.

[20] Wenqi Huang, Charley Lee, Leonard Tng, and Serena Ge. DeepSWE: Measuring frontier coding agents on original, long-horizon engineering tasks, 2026. arXiv:2607.07946.

[21] Kenneth Hung and William Fithian. Rank verification for exponential families. *The Annals of Statistics*, 47(2):758–782, 2019. doi: 10.1214/17-AOS1634.

[22] Shourya V. Jain. The leaderboard lottery: Why top LLM rankings are often coin flips, and how to fix them. Poster, ICML 2026 Workshop on Hypothesis Testing, 2026. URL https://icml.cc/virtual/2026/80012.

[23] Adel Javanmard and Andrea Montanari. Online rules for control of false discovery rate and false discovery exceedance. *The Annals of Statistics*, 46(2):526–554, 2018. doi: 10.1214/17-aos1559.

[24] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. arXiv:2310.06770.

[25] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter. *Transactions on Machine Learning Research*, 2025. arXiv:2407.01502.

[26] Ilmun Kim and Aaditya Ramdas. Locally minimax optimal confidence sets for the best model, 2025. arXiv:2503.21639.

[27] Anany Kotawala. Resolution diagnostics for paired LLM evaluation, 2026. arXiv:2605.30315. ICML 2026 Workshop on Hypothesis Testing.

[28] Jonas Kübler, Kailash Budhathoki, Matthäus Kleindessner, Xiong Zhou, Junming Yin, Ashish Khetan, and George Karypis. When LLMs get significantly worse: A statistical approach to detect model degradations. In *International Conference on Learning Representations*, 2026. arXiv:2602.10144.

[29] Jason D. Lee, Dennis L. Sun, Yuekai Sun, and Jonathan E. Taylor. Exact post-selection inference, with application to the lasso. *The Annals of Statistics*, 44(3):907–927, 2016.

[30] LessWrong contributor. Terminal-bench leaderboard rankings: Luck or skill? LessWrong post, https://www.lesswrong.com/posts/GiPmLmmbT6DyrwYkH/terminal-bench-leaderboard-rankings-luck-or-skill, 2026. Informal analysis; task-clustered bootstrap with a Benjamini–Hochberg correction over all pairs.

[31] Fengshuo Liu, Ying Liu, Ruize Sun, Lie Luo, and Siyuan Guo. Coding agents have converged: Why the SWE-bench leaderboard can no longer order its top entries, and what to measure instead, 2026. arXiv:2609.17394. To appear, ADMA 2026.

[32] Lovish Madaan, Aaditya K. Singh, Rylan Schaeffer, Andrew Poulton, Sanmi Koyejo, Pontus Stenetorp, Sharan Narang, and Dieuwke Hupkes. Quantifying variance in evaluation benchmarks, 2024. arXiv:2406.10229.

[33] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[34] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. arXiv:2601.11868.

[35] Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations, 2024. arXiv:2411.00640.

[36] Magne Mogstad, Joseph P. Romano, Azeem M. Shaikh, and Daniel Wilhelm. Inference for ranks with applications to mobility across neighbourhoods and academic achievement across countries. *The Review of Economic Studies*, 91(1):476–518, 2024. doi: 10.1093/restud/rdad006.

[37] Bitya Neuhof and Yuval Benjamini. Quantifying ranking uncertainty in LLM benchmarks, 2026. arXiv:2607.16259.

[38] Bitya Neuhof and Yuval Benjamini. Rank intervals for leaderboards: A hierarchical framework for model evaluation, 2026. arXiv:2606.08679.

[39] OpenAI. Introducing SWE-bench Verified. https://openai.com/index/introducing-swe-bench-verified/, 2024. Release note, August 2024; URL checked 2026-10-12.

[40] Andreas Petrou-Zeniou and Azeem M. Shaikh. Inference after ranking with applications to economic mobility, 2024. arXiv:2410.19212.

[41] Aaditya Ramdas, Fanny Yang, Martin J. Wainwright, and Michael I. Jordan. Online control of the false discovery rate with decaying memory. In *Advances in Neural Information Processing Systems*, 2017. arXiv:1710.00499.

[42] Rebecca Roelofs, Vaishaal Shankar, Benjamin Recht, Sara Fridovich-Keil, Moritz Hardt, John Miller, and Ludwig Schmidt. A meta-analysis of overfitting in machine learning. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[43] Moshe Shaked and J. George Shanthikumar. *Stochastic Orders*. Springer, New York, 2007. doi: 10.1007/978-0-387-34675-5.

[44] Shivalika Singh, Yiyang Nan, Alex Wang, Daniel D'Souza, Sayash Kapoor, Ahmet Üstün, Sanmi Koyejo, Yuntian Deng, Shayne Longpre, Noah A. Smith, Beyza Ermis, Marzieh Fadaee, and Sara Hooker. The leaderboard illusion, 2025. arXiv:2504.20879.

[45] SWE-bench team. SWE-bench Multilingual (dataset). Hugging Face dataset SWE-bench/SWE-bench_Multilingual, 2025. Accessed 2026-10-12.

[46] SWE-bench team. SWE-bench leaderboard submissions (repository). https://github.com/SWE-bench/experiments, 2026. Accessed 2026-10-12.

[47] Terminal-Bench contributors. Terminal-bench 2 leaderboard submissions (dataset). https://huggingface.co/datasets/harborframework/terminal-bench-2-leaderboard, 2026. Accessed 2026-10-12.

[48] Jinjin Tian and Aaditya Ramdas. Online control of the familywise error rate. *Statistical Methods in Medical Research*, 30(4):976–993, 2021. doi: 10.1177/0962280220983381.

[49] Naman Vats and Oleg Golev. The scaffold effect in coding agents: Harness choice as a hidden variable in coding-agent evaluation, 2026. arXiv:2607.22585.

[50] Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Benjamin Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Sreemanti Dey, Shubh-Agrawal, Sandeep Singh Sandha, Siddartha Naidu, Chinmay Hegde, Yann LeCun, Tom Goldstein, Willie Neiswanger, and Micah Goldblum. LiveBench: A challenging, contamination-limited LLM benchmark. In *International Conference on Learning Representations*, 2025. arXiv:2406.19314.

[51] Brian L. Wiens. A fixed sequence Bonferroni procedure for testing multiple endpoints. *Pharmaceutical Statistics*, 2(3):211–215, 2003. doi: 10.1002/pst.64.

[52] Chen Yang and Jun Chen. How sensitive are LLM leaderboard claims to hidden model selection?, 2026. arXiv:2609.28177.

[53] John Yang, Carlos E. Jimenez, Alex L. Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R. Narasimhan, Diyi Yang, Sida I. Wang, and Ofir Press. SWE-bench Multimodal: Do AI systems generalize to visual software domains?, 2024. arXiv:2410.03859.

[54] Yunbei Zhang, Janet Wang, Yingqiang Ge, Weijie Xu, Jihun Hamm, and Chandan K. Reddy. Stop comparing LLM agents without disclosing the harness, 2026. arXiv:2605.23950.

[55] Tijana Zrnic and William Fithian. Locally simultaneous inference. *The Annals of Statistics*, 52(3):1227–1253, 2024. doi: 10.1214/24-aos2391.

[56] Tijana Zrnic and William Fithian. A flexible defense against the winner's curse. *The Annals of Statistics*, 53(6):2516–2535, 2025. doi: 10.1214/25-aos2553.

A Reporting checklist for leaderboards

1. Publish per-task outcomes for every submission, in one table with the same task identifiers.
2. Publish the full task list, including tasks that no submission solved.
3. State the number of runs behind each score and the trial structure.
4. State the number of variants, prompts or configurations tried against the benchmark before the submission.
5. Order same-date submissions by an explicit rule (for example submission time).
6. Report each record's margin over the previous record with a paired interval, not only the score.
7. Mark a submission as a new state of the art only if it beats the previous record in a selection-aware paired test at a stated level, and say which test.
8. Report the benchmark's resolution: the smallest difference it detects with 80% power, and the number of tasks needed to resolve a one-point gap.
9. Report a confidence set of the best system next to the ranking.
10. Report the selection-corrected score of the current leader, and say that tuning against public tasks is not covered by the correction.

B What a maintainer sees

The package answers the questions of this paper for one submission in one call. Below is its output, unedited, for the current SWE-bench Verified leader; the `report` command prints the same facts for the whole board, and `pr-check` prints them as a Markdown comment for a pull request. The leader ties the previous leader exactly (difference 0.0 points, so it set no record), and the board's latest record (the last row of Table 12) beat the record it replaced by just two tasks, for which the selective p-value is exactly one: the smallest possible record carries no evidence at all.

```
$ python -m honestboard check swebench_verified.json 20251215_livesweagent_claude-opus-4-5
20251215_livesweagent_claude-opus-4-5: score 79.2%, observed rank 1, 95% rank interval (1.0,
    6.0), top-rank freq.=0.35
Inside the 95% confidence set of the best: True (set size 14)
Paired test, 20251215_livesweagent_claude-opus-4-5 vs
    20251205_sonar-foundation-agent_claude-opus-4-5: difference +0.0 pts, one-sided exact
    p=0.566
Selection-adjusted score: 78.1%, 95% interval [74.9%, 81.5%] (reported 79.2%)
```

C Proof of Lemma 1

Write $T_n(k) = \sum_{j \geq k} \binom{n}{j}$, so that $p_{\mathrm{sel}}(k, n) = T_n(k)/T_n(\lfloor n/2 \rfloor + 1)$ for a challenger with $k > n/2$ wins among n discordant tasks. Pascal's rule gives $\sum_{j \geq k+1} \binom{n+1}{j} = 2T_n(k) - \binom{n}{k}$ and $\sum_{j \geq k} \binom{n+1}{j} = 2T_n(k) + \binom{n}{k-1}$. The ratio $T_n(k)/\binom{n}{k} = 1 + r_k + r_k r_{k+1} + \cdots$ with $r_j = (n-j)/(j+1)$ decreasing in j is decreasing in k, and so is $T_n(k)/\binom{n}{k-1} = r_{k-1} + r_{k-1} r_k + \cdots$.

A challenger win is added $(k, n \to k+1, n+1)$. For even $n = 2m$ the denominators are $T_n(m+1)$ and $2T_n(m+1) + \binom{n}{m}$, and the claim $\{2T_n(k) - \binom{n}{k}\}/\{2T_n(m+1) + \binom{n}{m}\} \leq T_n(k)/T_n(m+1)$ reduces to $-\binom{n}{k} T_n(m+1) \leq T_n(k)\binom{n}{m}$, which holds. For odd $n = 2m+1$ the denominators are $T_n(m+1)$ and $2T_n(m+1) - \binom{n}{m+1}$, and the claim reduces to $T_n(k)/\binom{n}{k} \leq T_n(m+1)/\binom{n}{m+1}$ for $k \geq m+1$, which is the monotonicity of the first ratio.

A holder win is added $(k, n \to k, n+1)$. For even $n = 2m$ the claim $\{2T_n(k) + \binom{n}{k-1}\}/\{2T_n(m+1) + \binom{n}{m}\} \geq T_n(k)/T_n(m+1)$ reduces to $T_n(k)/\binom{n}{k-1} \leq T_n(m+1)/\binom{n}{m}$ for $k \geq m+1$, the monotonicity of the second ratio. For odd n it reduces to $\binom{n}{k-1} T_n(m+1) + T_n(k)\binom{n}{m+1} \geq 0$. □

D A normal-theory selective p-value for a record step

With $\widehat{p} \sim N(\theta, \widehat{\Sigma})$, the event that b is a record at position k and a was the holder is the polyhedron $\{\widehat{p}_b > \widehat{p}_a,\ \widehat{p}_a \geq \widehat{p}_j\ \forall j < k\}$, and the polyhedral lemma [29] gives the truncated-normal law of the contrast $\widehat{p}_b - \widehat{p}_a$ on it, hence a one-sided selective p-value for $\theta_b \leq \theta_a$. Scores are pass counts over T, so they sit on a grid of spacing $1/T$, and the holder is often tied or one task apart from its closest earlier competitor. Without a correction that puts the statistic right on the edge of the cut-off region and the p-value collapses: in our simulations it wrongly rejected 14% of null steps at the 5% level even with the true covariance. Relaxing each constraint by half a grid step and taking the tail from half a step below the observed statistic (a continuity correction) brings the rate to 1 to 3%. We have no finite-sample guarantee for the corrected version and report it only as a comparison (Tables 8 and 13).